Machine Learning Optimisation: Objectives, Algorithms, and Diagnostics

Mick Cooney

18 October 2026

Table of Contents

1 Orientation, Scope, and Notation **3**
1.1 Purpose, Scope, and Reading Route . . . 3
1.2 What This Primer Is Not . . . 5
1.3 Assumed Background . . . 6
1.4 Notation . . . 8

2 From a Learning Task to an Optimisation Problem **11**
2.1 Population Risk and Empirical Risk . . . 11
2.2 Parameters, Hyperparameters, and State . . . 13
2.3 Recurring Example: Regularised Logistic Regression . . . 14
2.4 Objective, Metric, and Constraint Are Different Roles . . . 21
2.5 Worked Formulation Audit: Weighted Logistic Regression . . . 22

3 Geometry, Derivatives, and Conditioning **22**
3.1 Directional Derivatives, Gradients, Jacobians, and Hessians . . . 23
3.2 Smoothness and Curvature Bounds . . . 25
3.3 Recurring Example: A Rotated Ill-Conditioned Quadratic . . . 25
3.4 Derivative Checks and Numerical Limits . . . 32
3.5 Automatic Differentiation for Optimisation . . . 32
3.5.1 The Derivative Object the Solver Needs . . . 32
3.5.2 From a Written Loss to an Executable Objective . . . 34
3.5.3 Memory, State, and Non-Smooth Operations . . . 34
3.5.4 Worked Derivative Products . . . 35
3.5.5 Validate AD Independently . . . 37
3.6 Conditioning Beyond One Number . . . 38

4 Problem Classes and Convergence Claims **38**
4.1 Classify Before You Optimise . . . 39
4.2 What Common Rates Actually Say . . . 41
4.3 Stopping Criteria Are Claims About Evidence . . . 43
4.4 A Guarantee Checklist . . . 44

4.5 Reading a Convergence Theorem as an Engineering Contract . . . 45

5 First-Order Methods **46**
5.1 Gradient Descent and Line Search . . . 46
5.2 Stochastic and Mini-Batch Gradients . . . 47
5.3 Momentum and Accelerated Methods . . . 50
5.4 Learning-Rate Schedules . . . 51
5.5 Adaptive Diagonal Methods . . . 52
5.6 Coordinate, Subgradient, and Proximal Methods . . . 53
5.7 First-Order Selection Summary . . . 55
5.8 Choosing Batch Size, Step Size, and Schedule Together . . . 55
5.9 Online Regret and Offline Optimisation . . . 56

6 Curvature-Aware and Structured Methods **57**
6.1 Newton's Method and Safeguards . . . 57
6.2 Gauss–Newton and Natural-Gradient Ideas . . . 59
6.3 Trust-Region Methods . . . 60
6.4 Quasi-Newton Methods . . . 60
6.5 Hessian-Vector Products and Preconditioning . . . 61
6.6 Curvature-Aware Selection Summary . . . 63
6.7 A Worked Curvature-Aware Decision . . . 63
6.8 Model Quality and Step Quality . . . 64

7 Constraints, Duality, and Regularisation **64**
7.1 Feasible Sets and Projections . . . 64
7.2 Lagrangians, KKT Conditions, and Duality . . . 65
7.3 Penalty, Barrier, and Augmented-Lagrangian Methods . . . 67
7.4 Regularisation as Geometry and Statistical Choice . . . 68
7.5 Constraints and Regularisation Failure Modes . . . 70
7.6 Worked KKT Example: Projection Onto a Simplex . . . 70
7.7 Choosing a Constrained Method . . . 71

8 Non-Convex Optimisation in Machine Learning **71**
8.1 What the Guarantees Become . . . 72
8.2 Saddles, Plateaux, Symmetries, and Cliffs . . . 73
8.3 Mini-Batch Noise and Training Dynamics . . . 74
8.4 Optimiser Choice and Generalisation . . . 75
8.5 Hyperparameter Search as an Outer Loop . . . 76
8.6 A Defensible Non-Convex Claim . . . 77

9 Diagnostics and an End-to-End Workflow **77**
9.1 Step 1: Freeze the Optimisation Contract . . . 78
9.2 Step 2: Validate the Objective and Derivatives . . . 79
9.3 Step 3: Run Small Problems With Known Answers . . . 81
9.4 Step 4: Monitor Quantities That Separate Failures . . . 82
9.5 Step 5: Compare Methods Fairly . . . 84

9.6 Step 6: Decide, Stop, and Preserve Evidence 85
9.7 Method-Selection Checklist 86

10 Further Reading **88**

1 Orientation, Scope, and Notation

1.1 Purpose, Scope, and Reading Route

Optimisation is the part of machine learning that turns a model, data, and a loss into fitted parameters. The optimiser does not decide whether the model is useful, whether the data represent the deployment population, or whether the reported metric answers the real question. It solves, or approximately solves, the mathematical problem it is given. This primer develops the judgement needed to formulate that problem, select an algorithm, interpret its guarantees, and diagnose a run that has gone wrong.

The main route begins with a learning task and derives an objective. It then connects derivatives to local geometry, separates important problem classes, and develops first-order, curvature-aware, constrained, proximal, and stochastic methods. The final sections explain what changes for non-convex machine learning, how hyperparameter search differs from parameter optimisation, and how to build an evidence trail around an optimisation run. The emphasis is continuous optimisation, because that is where gradients, automatic differentiation, and large-scale learning meet.

The central question is not "which optimiser is fastest?" It is "what mathematical structure does this problem expose, and what evidence would show that an algorithm has used that structure safely?" A method with a strong theorem can still be a poor engineering choice if its assumptions are violated, its linear algebra dominates the run, or its stopping rule measures the wrong quantity. A practical method can be entirely appropriate when its empirical behaviour is monitored and its limits are stated. The primer therefore treats guarantees and diagnostics as a pair: a rate explains what should happen under stated conditions, while a diagnostic checks whether the run resembles that regime.

The document follows the lifecycle of an optimisation result. First, separate population risk, empirical risk, regularisation, constraints, validation objectives, and hyperparameters. Next, identify the derivative and curvature information

that is available, including whether it is exact, automatic, noisy, subsampled, or only approximately solved. Then classify the problem as smooth or non-smooth, convex or non-convex, unconstrained or constrained, deterministic or stochastic, and well-conditioned or ill-conditioned. Only after that classification should a method be selected. Finally, record the objective values, gradient or stationarity measures, constraint violations, validation results, step sizes, random state, and termination reason so that the result can be reproduced and challenged.

Two synthetic examples recur throughout. The first is a rotated, ill-conditioned quadratic whose optimum is known exactly. It makes conditioning, zig-zagging, preconditioning, momentum, and Newton steps visible without statistical complications. The second is regularised binary logistic regression. It connects empirical risk, probabilistic prediction, regularisation, stochastic gradients, Hessian structure, and validation diagnostics in one convex machine-learning problem. Both examples are small enough to inspect by hand; neither is a performance benchmark.

These examples are deliberately complementary. The quadratic isolates geometry: if an iteration is slow, oscillatory, or unstable, the objective and its spectrum make the reason visible. Logistic regression then adds the concerns that make machine-learning optimisation different from a textbook quadratic: a finite sample, a statistical loss, a regularisation choice, minibatch noise, numerical overflow, and a distinction between training and held-out evidence. The examples are not presented as recipes to copy into production. They are controlled experiments for learning how to ask whether an observed optimisation trace is plausible.

You can read the primer in three ways. For a mathematical route, read Sections 2–7 in order. For a training-debugging route, read Sections 2–4, then Sections 8–9. For a method-selection route, begin with the guarantee table in Section 4, compare Sections 5–7, and finish with the workflow in Section 9. The Numerical Analysis primer supplies broader numerical context, while the Deep Learning primer develops neural architectures and backpropagation.

Each route has a different stopping point. A reader who needs to understand a convergence claim can stop after the problem-class and method sections, provided the assumptions are recorded. A reader debugging a training run should continue to the diagnostics section even if the chosen optimiser is conventional. A reader designing a benchmark should read the workflow and the

discussion of validation leakage: comparing methods on a repeatedly reused validation set can turn the benchmark itself into an uncontrolled outer optimisation loop. The intended outcome is not memorisation of algorithm names, but a defensible account of what was optimised, under which assumptions, and why the reported result is credible.

1.2 What This Primer Is Not

This is not a proof-based optimisation textbook, a catalogue of solver APIs, or a claim that one optimiser is best across machine-learning tasks. Proof sketches are included where they expose an assumption or a failure mode, but the Further Reading section points to complete treatments. The core calculations use the Python standard library so that their assumptions remain visible; the automatic-differentiation section adds explicitly labelled Python, R, and Julia examples that require the stated packages. Production work should use tested numerical kernels and solver implementations.

The primer does not re-derive automatic differentiation, floating-point analysis, linear-system algorithms, or neural-network architectures. It also does not cover discrete or mixed-integer optimisation, optimal control, reinforcement-learning policy optimisation, or a complete theory of derivative-free global search. Hyperparameter optimisation appears only far enough to separate the outer model-selection problem from the inner parameter-fitting problem.

The boundary with the neighbouring primers is intentional. Numerical Analysis develops the broader numerical toolkit: floating-point error, linear algebra, root finding, automatic differentiation, and an introductory optimisation workflow. Deep Learning explains neural architectures, forward computation, backpropagation, and the major families of learned models. This primer assumes those subjects where needed and concentrates on the optimisation layer that connects them: objective formulation, local geometry, problem classification, method assumptions, convergence language, and evidence from an actual run. Repeating a definition here is useful only when its optimisation consequence is different from its numerical or modelling consequence.

Nor is this a framework tutorial. The examples name algorithmic families rather than recommending a particular library, version, device, or cloud service. A production implementation must still establish precision, batching, parallelism, memory use, checkpoint semantics, and deterministic or stochastic execution.

Those choices can change the effective optimisation problem. The document gives the questions and tests that expose those changes, not a universal configuration file.

An optimisation result is not automatically a statistical guarantee. A small training gradient does not establish good generalisation, causal validity, calibration, fairness, robustness, or safe deployment. Conversely, a model can have useful validation performance before the training objective is solved accurately. The workflow therefore keeps optimisation evidence, statistical evidence, and operational acceptance criteria separate.

There are also limits on what a convergence statement can establish. "Converges" may mean that objective error tends to zero, that the gradient norm becomes small, that iterates approach a solution set, or only that a randomly selected iterate has a bounded expected stationarity measure. The conclusion may concern an exact arithmetic sequence rather than a finite-precision implementation, and it may require a fixed step size, a bounded variance, a Lipschitz gradient, a convex feasible set, or a positive-definite local model. Those qualifications are part of the result. They are not footnotes to be removed when communicating a benchmark.

The primer therefore treats failure as information. Divergence can indicate a step-size error, an indefinite curvature model, a broken gradient, or a scale mismatch. A flat training curve can indicate convergence, saturation, a data pipeline problem, or a metric that hides progress. An excellent validation score can indicate a useful model, a lucky split, or leakage. The final workflow asks the reader to distinguish these possibilities with measurements rather than attributing every outcome to the optimiser's brand name.

1.3 Assumed Background

You should be comfortable with vectors, matrices, matrix multiplication, derivatives of scalar functions, and elementary probability. The primer defines optimisation-specific terms such as smoothness, strong convexity, stationarity, duality, and conditioning before using their guarantees. Familiarity with Python helps with the core executable examples; the package-specific automatic-differentiation examples name their R and Julia dependencies explicitly.

The most useful preparation is operational rather than encyclopaedic. You

should be able to read a scalar objective, identify the shape of every term, and distinguish a vector from a scalar-valued function of a vector. You should know that a gradient is a direction in parameter space, a Jacobian maps perturbations of one vector to perturbations of another, and a Hessian describes the local change of the gradient. You need not derive every identity from first principles, but you should be willing to check dimensions and test a derivative numerically.

You should also know the basic statistical distinction between a quantity computed on the training sample and a quantity defined under a population distribution. An expectation, a sample average, a minibatch estimate, and a validation score answer related but different questions. Stochastic optimisation uses random estimates of derivatives; it does not make the objective itself random by magic, and it does not turn a validation score into an unbiased estimate merely because a random seed was used.

Finally, you should be able to read a short iterative algorithm and reason about its state. At a minimum, identify the current parameters, the gradient estimate, the step size, any momentum or preconditioner state, and the termination test. For constrained or proximal methods, identify the feasible set or regulariser and the operation that enforces it. For a non-convex problem, distinguish a stationary point from a global minimiser. These distinctions are more important here than fluency with a specific software interface.

The prerequisite map below identifies the concepts used and where to refresh them.

Area	Needed Here	Suggested Route
Linear algebra	Inner products, norms, eigenvalues, positive-definite matrices, linear systems	Numerical Analysis: Linear Algebra Foundations
Multivariable calculus	Partial derivatives, chain rule, gradients, Jacobians, Hessians	Numerical Analysis: Automatic Differentiation
Probability	Expectations, independent sampling, variance, conditional probability	Information Theory: Probability Foundations

Area	Needed Here	Suggested Route
Machine learning	Training and validation splits, losses, backpropagation, regularisation	Deep Learning: Training Deep Networks
Reproducible computation	Environment, seeds, logs, artefacts, and safe Git workflow	Linux, Git, and Reproducible Developer Workflows

If eigenvalues are unfamiliar, read the linear-algebra material before the conditioning section. If probability is the gap, focus on expectation and variance before stochastic gradients. You do not need measure theory, but you do need to distinguish a finite-sample average from an expectation under an unknown population distribution.

A quick readiness check is useful. Given a proposed update, can you say whether the update has the same dimension as the parameter vector? Given a Hessian, can you say whether it is symmetric, positive definite, indefinite, or singular at the point of interest? Given a minibatch gradient, can you name the population or empirical quantity it estimates and the source of its variance? Given a reported tolerance, can you say whether it bounds objective error, gradient norm, constraint violation, or only a change between successive iterates? If not, read the relevant prerequisite before interpreting a convergence plot.

The notation in the primer is intentionally explicit about these distinctions. Bold symbols denote vectors or matrices, a function argument shows which quantity is being differentiated, and subscripts identify examples or iterations rather than silently changing the object being discussed. The same convention carries into the code: arrays have named roles, intermediate quantities are not hidden behind a framework call, and numerical output is treated as evidence that can be checked.

1.4 Notation

Vectors and matrices are bold. For Latin symbols this is written with `\mathbf`; for Greek symbols, whose bold forms are not reliably produced by `\mathbf` in every TeX renderer, this primer uses `\boldsymbol`. A superscript $^\top$ denotes trans-

pose. All vector norms are Euclidean unless another norm is named, and matrix norms are the corresponding induced norms. The same letter may receive a local subscript for a training example or iteration, but the underlying meaning stays fixed. The HTML build is checked in both MathJax and MathML modes because a notation that looks acceptable in one output is not automatically portable to the other.

Scalars are usually italic lower-case or upper-case letters, vectors are bold lower-case letters, and matrices are bold upper-case letters. A scalar-valued function such as $F(\theta)$ takes a vector argument but returns one number. Its gradient $\nabla F(\theta)$ is a vector, and its Hessian $\nabla^2 F(\theta)$ is a matrix. A vector-valued residual $r(\theta)$ has a Jacobian $\mathbf{J}_r(\theta)$ whose rows correspond to output components and columns correspond to parameters. Keeping those shapes visible prevents many transposition and aggregation errors.

The symbols F, ℓ, R, and $\mathcal{R}$ have distinct roles. ℓ is normally a per-example loss, F is the finite objective presented to the optimiser, R is a regularisation function, and $\mathcal{R}$ is population risk. A validation loss may be written with a separate subscript rather than silently reusing F. The distinction matters whenever a method is claimed to improve one quantity while the experiment measures another.

An inner product is written $\mathbf{u}^\top \mathbf{v}$, and $\|\mathbf{u}\|$ without a subscript means the Euclidean norm. The induced operator norm is written $\|\mathbf{A}\|$ when the context is clear. Subscripts such as $\|\mathbf{u}\|_1$ and $\|\mathbf{u}\|_\infty$ name different geometries; they are not decorative alternatives. The condition number $\kappa = L/\mu$ later refers to a smooth strongly convex objective, not automatically to a matrix condition number in an unrelated norm.

The table introduces the symbols used throughout.

Symbol	Meaning	First Use
$\theta \in \mathbb{R}^d$	Model parameter vector with d components	Learning objective
$z_i = (\mathbf{x}_i, y_i)$	Training example with features and target	Empirical risk
$\ell(\theta; z_i)$	Per-example loss	Empirical risk
$F(\theta)$	Objective being minimised	Learning objective

Symbol	Meaning	First Use
$R(\theta)$	Regularisation function	Regularised risk
$\lambda \geq 0$	Regularisation strength	Regularised risk
$\nabla F(\theta)$	Gradient column vector	First-order geometry
$\nabla^2 F(\theta)$	Hessian matrix	Curvature
$\mathbf{J}_r(\theta)$	Jacobian of a vector-valued residual r	Nonlinear least squares
$\alpha_k > 0$	Step size at iteration k	Gradient descent
$\mathbf{g}_k$	Exact or estimated gradient used at iteration k	Iterative methods
$L > 0$	Gradient Lipschitz constant, or smoothness constant	Smooth objectives
$\mu > 0$	Strong-convexity constant	Strong convexity
$\kappa = L/\mu$	Condition number for a smooth strongly convex objective	Conditioning
$\mathcal{C}$	Feasible set	Constrained optimisation
$\Pi_{\mathcal{C}}$	Euclidean projection onto $\mathcal{C}$	Projected gradient
$\text{prox}_{\alpha R}$	Proximal operator of αR	Proximal gradient
$\mathbb{E}$, Var	Expectation and variance	Stochastic gradients
$\varepsilon > 0$	Stated numerical tolerance	Stopping rules

An iteration index is $k = 0, 1, 2, \ldots$. A star marks an optimiser or optimal value, such as θ^* and F^*. The symbol arg min denotes the set of minimisers; writing a single θ^* assumes that a particular minimiser has been selected, not necessarily that it is unique.

The notation $a_k = O(b_k)$ describes an asymptotic upper bound up to a constant under the assumptions stated with the result. A rate for objective error does not

automatically imply the same rate for parameter error, gradient norm, validation loss, or wall-clock time.

There are three recurring layers of notation. Problem data such as z_i, $\mathbf{x}_i$, and y_i are fixed for a stated experiment. Parameters such as θ are the variables being optimised. Algorithm state such as $\mathbf{g}_k$, momentum buffers, inverse-curvature approximations, and random-number state changes from iteration to iteration. A checkpoint that stores parameters but omits relevant state may reproduce a model but not reproduce the optimisation trajectory.

The notation for a derivative also carries a promise about what is held fixed. $\nabla F(\theta)$ differentiates the stated objective with respect to its parameter argument. A partial derivative may hold hyperparameters, data, or a constraint multiplier fixed. In a bilevel problem, $\theta^*(\eta)$ is itself a function of the outer variable η; treating it as constant changes the outer derivative. Later sections call out these distinctions when they affect an algorithm or a guarantee.

Displayed equations use the same symbols as the executable examples. When a formula is implemented, the implementation should preserve the mathematical grouping: stable log-loss expressions should not form probabilities and then take logarithms at extreme scores; a Hessian-vector product should not be confused with a materialised Hessian; and a stochastic gradient should not be described as exact merely because automatic differentiation produced it. These are notation and semantics checks as much as coding checks.

Verified against primary sources for all sections; unverified claims are flagged inline.

2 From a Learning Task to an Optimisation Problem

Optimisation begins before an algorithm is selected. You must decide what parameters are free, what data enter the objective, which errors receive which costs, what constraints apply, and what evidence will be held out. A poor formulation solved perfectly is still a poor result.

2.1 Population Risk and Empirical Risk

Let Z be an example drawn from an unknown population distribution P. For a model parameterised by θ and a loss ℓ, the population risk is

$$\mathcal{R}(\theta) = \mathbb{E}_{Z \sim P}\big[\ell(\theta; Z)\big].$$

The expectation is usually unavailable. Given training examples $z_1, z_2, \dots, z_n$, empirical risk minimisation replaces it with an average and often adds a regulariser:

$$F(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell(\theta; z_i) + \lambda R(\theta).$$

This finite objective is the optimiser's problem. Its relationship to population performance depends on the sampling process, model class, loss, regularisation, and validation design. Independence and identical distribution are useful assumptions for many statistical results, but an optimiser can minimise the empirical objective whether or not those assumptions hold. It cannot diagnose that the sample is biased merely from the objective value.

The scaling convention matters. Using a mean loss keeps the data term on roughly the same scale as n changes; using a sum changes the numerical meaning of λ. Two implementations that differ only by this convention can report the same model family but solve different objectives unless the regularisation coefficient is rescaled.

The average also exposes *finite-sum structure*. If

$$F(\theta) = \frac{1}{n} \sum_{i=1}^{n} f_i(\theta) + \lambda R(\theta),$$

then a full gradient costs one contribution from every example, while a stochastic method can estimate the data term from a subset. A coordinate method may instead exploit sparsity in the features, and a variance-reduced method may store or periodically recompute information about the individual f_i. Writing only "minimise loss" hides the structure that controls both cost and variance.

Weights change more than the relative importance of examples. For non-negative weights w_i, a common convention is

$$F_w(\theta) = \frac{1}{\sum_i w_i} \sum_{i=1}^{n} w_i f_i(\theta) + \lambda R(\theta).$$

Normalising by $\sum_i w_i$ keeps the data term invariant if every weight is multiplied by the same positive constant. Dividing by n does not. Either convention can be valid, but the optimiser, regularisation coefficient, and reported objective must use the same one. Class-balanced sampling introduces a related distinction: sampling minority examples more often without an importance correction changes the objective, while sampling with the appropriate correction can preserve the original weighted expectation at the cost of higher variance.

Not every training objective is a separable average. Pairwise ranking losses couple two examples; contrastive losses can couple every item in a batch through a shared denominator; normalisation layers can make one example's contribution depend on the other examples in the batch. In those cases, "the mini-batch gradient is an unbiased estimate of the full gradient" needs proof for the implemented batching scheme. It is not implied by the presence of a data loader.

The regulariser also has an averaging convention. If a program adds λR once per batch and then averages those batch losses over an epoch, the effective penalty depends on the number of batches unless the bookkeeping is correct. A robust objective test duplicates the data, changes the batch partition while preserving the same examples, and checks whether the intended mathematical objective and its full gradient remain unchanged.

2.2 Parameters, Hyperparameters, and State

Parameters are variables changed by the inner optimiser, such as regression coefficients or neural-network weights. Hyperparameters configure the learning procedure or model class, such as λ, a network depth, or a learning-rate schedule. Optimiser state includes momentum buffers, running squared-gradient estimates, counters, and random-number state. A checkpoint that stores only parameters is insufficient to resume a stateful optimiser exactly.

Hyperparameter selection is an outer optimisation problem. If A_η trains parameters under hyperparameters η, a simplified bilevel formulation is

$$\theta^*(\eta) \in \arg\min_{\theta} F_{\text{train}}(\theta; \eta), \qquad \min_{\eta} F_{\text{valid}}(\theta^*(\eta)).$$

The validation objective is not another term to optimise repeatedly while still calling the same data "held out". Reusing validation results to choose many configurations makes the validation set part of the selection procedure. A final test set or a nested resampling design is needed when you want an unbiased assessment of that procedure.

2.3 Recurring Example: Regularised Logistic Regression

For binary targets $y_i \in \{0, 1\}$ and feature vectors $\mathbf{x}_i \in \mathbb{R}^d$, logistic regression models

$$p_i(\theta) = \frac{1}{1 + \exp(-\mathbf{x}_i^\top \theta)}.$$

With an ℓ_2 penalty, the objective is

$$F(\theta) = \frac{1}{n} \sum_{i=1}^{n} [-y_i \log p_i - (1 - y_i) \log(1 - p_i)] + \frac{\lambda}{2} \|\theta\|^2.$$

The stable computational form of the per-example loss, with score $s_i = \mathbf{x}_i^\top \theta$, is $\max(s_i, 0) - y_i s_i + \log(1 + \exp(-|s_i|))$. It avoids overflow and cancellation caused by forming probabilities and then taking logarithms at extreme scores.

The gradient and Hessian are

$$\nabla F(\theta) = \frac{1}{n} \mathbf{X}^\top (\mathbf{p} - \mathbf{y}) + \lambda \theta,$$

and

$$\nabla^2 F(\theta) = \frac{1}{n} \mathbf{X}^\top \mathbf{W} \mathbf{X} + \lambda \mathbf{I},$$

where $\mathbf{W}$ is diagonal with entries $p_i(1 - p_i)$. The matrix $\mathbf{W}$ is positive semidefinite, so the unregularised objective is convex. If $\lambda > 0$ and every coefficient is

penalised, the Hessian is positive definite and the objective is strongly convex. An intercept is often left unpenalised; then strong convexity requires a separate rank argument.

This example exposes a clean separation of concerns.

This experiment constructs a deterministic binary-classification fixture without a random-number generator. It takes the first 48 observations as training data and the remaining 24 as validation data, estimates feature means and scales from the training partition only, and fixes every optimisation choice before validation is evaluated. The objective is mean binary log loss plus an ℓ_2 penalty with $\lambda = 0.1$ on all three coefficients, including the intercept. Full-batch gradient descent uses the explicit global bound $L_{\text{bound}} = \frac{1}{4n}\sum_i \|\mathbf{x}_i\|^2 + \lambda$ and step $1/L_{\text{bound}}$.

The code uses stable softplus and sigmoid calculations, asserts monotone training-objective descent, checks the analytic gradient by a directional finite difference, tests invariance to training-row order, and reports held-out log loss, accuracy at the declared 0.5 threshold, and Brier score only after fitting. The validation figures describe this one fixed split; they are not uncertainty intervals or evidence of population generalisation.

Python:

```python
from math import exp, hypot, isfinite, log1p, sqrt


LAMBDA = 0.1
ITERATIONS = 60
CHECKPOINTS = {0, 1, 2, 5, 10, 20, 50, 60}


def raw_observation(i):
    # Integer recurrences make the fixture identical across languages.
    x1 = (((37 * i + 11) % 101) - 50) / 15.0
    x2 = (((53 * i + 7) % 97) - 48) / 15.0
    noise = (((29 * i + 3) % 19) - 9) / 4.0
    y = int(0.3 + 1.2 * x1 - 0.9 * x2 + noise >= 0.0)
    return x1, x2, y


raw = [raw_observation(i) for i in range(72)]
raw_train, raw_validation = raw[:48], raw[48:]
means = tuple(sum(row[j] for row in raw_train) / len(raw_train)
              for j in (0, 1))
scales = tuple(sqrt(sum((row[j] - means[j]) ** 2 for row in raw_train)
                    / len(raw_train)) for j in (0, 1))


def prepare(rows):
    x = [(1.0, (row[0] - means[0]) / scales[0],
          (row[1] - means[1]) / scales[1]) for row in rows]
```

```
    y = [row[2] for row in rows]
    return x, y


X_TRAIN, Y_TRAIN = prepare(raw_train)
X_VALIDATION, Y_VALIDATION = prepare(raw_validation)


def dot(left, right):
    return sum(a * b for a, b in zip(left, right))


def softplus(score):
    return max(score, 0.0) + log1p(exp(-abs(score)))


def sigmoid(score):
    if score >= 0.0:
        return 1.0 / (1.0 + exp(-score))
    value = exp(score)
    return value / (1.0 + value)


def data_loss(theta, x, y):
    return sum(softplus(dot(row, theta)) - target * dot(row, theta)
               for row, target in zip(x, y)) / len(y)


def objective_gradient(theta, x=X_TRAIN, y=Y_TRAIN):
    probabilities = [sigmoid(dot(row, theta)) for row in x]
    gradient = [sum((p - target) * row[j]
                    for row, target, p in zip(x, y, probabilities)) / len(y)
                + LAMBDA * theta[j] for j in range(3)]
    penalty = 0.5 * LAMBDA * dot(theta, theta)
    loss = data_loss(theta, x, y)
    return loss + penalty, loss, penalty, tuple(gradient)


def metrics(theta, x, y):
    probabilities = [sigmoid(dot(row, theta)) for row in x]
    loss = data_loss(theta, x, y)
    accuracy = sum((p >= 0.5) == bool(target)
                   for p, target in zip(probabilities, y)) / len(y)
    brier = sum((p - target) ** 2
                for p, target in zip(probabilities, y)) / len(y)
    return loss, accuracy, brier


# Since sigma(s)(1-sigma(s)) <= 1/4, this trace bound is a valid global
# Lipschitz bound for the penalised gradient. It is conservative but explicit.
smoothness_bound = (sum(dot(row, row) for row in X_TRAIN)
                    / (4.0 * len(X_TRAIN)) + LAMBDA)
step_size = 1.0 / smoothness_bound
theta = (0.0, 0.0, 0.0)
trace = []
for k in range(ITERATIONS + 1):
    value, loss, penalty, gradient = objective_gradient(theta)
    trace.append((k, theta, value, loss, penalty, hypot(*gradient)))
```

```
    if k < ITERATIONS:
        theta = tuple(coef - step_size * grad
                      for coef, grad in zip(theta, gradient))

# The checks distinguish implementation correctness from held-out behaviour.
values = [row[2] for row in trace]
assert all(right <= left + 1e-14
           for left, right in zip(values, values[1:]))
assert trace[-1][5] < 1e-8
assert all(isfinite(item) for row in trace for item in row[2:])
probe = (0.2, -0.3, 0.4)
direction = (0.5, -0.4, sqrt(0.59))
h = 1e-6
plus = tuple(value + h * delta for value, delta in zip(probe, direction))
minus = tuple(value - h * delta for value, delta in zip(probe, direction))
finite_difference = ((objective_gradient(plus)[0]
                      - objective_gradient(minus)[0]) / (2.0 * h))
analytic = dot(objective_gradient(probe)[3], direction)
relative_error = abs(finite_difference - analytic) / max(1.0, abs(analytic))
assert relative_error < 1e-8
reversed_result = objective_gradient(probe, tuple(reversed(X_TRAIN)),
                                     tuple(reversed(Y_TRAIN)))
original_result = objective_gradient(probe)
assert abs(reversed_result[0] - original_result[0]) < 1e-15
assert max(abs(a - b)
           for a, b in zip(reversed_result[3], original_result[3])) < 1e-15
assert isfinite(softplus(1000.0)) and isfinite(softplus(-1000.0))

print(f"regularised logistic: train=48 validation=24 lambda={LAMBDA:.3f}")
print(f"global smoothness bound={smoothness_bound:.6f} step={step_size:.6f}")
print("step     objective      data_loss        penalty     "
      "|gradient|       |theta|")
for k, coefficients, value, loss, penalty, grad_norm in trace:
    if k in CHECKPOINTS:
        print(f"{k:4d}  {value:13.8f}  {loss:13.8f}  {penalty:13.8f}"
              f"  {grad_norm:13.6e}  {hypot(*coefficients):13.6e}")

train_loss, train_accuracy, train_brier = metrics(theta, X_TRAIN, Y_TRAIN)
validation_loss, validation_accuracy, validation_brier = metrics(
    theta, X_VALIDATION, Y_VALIDATION)
print("final theta=(" + ", ".join(f"{value:.6f}" for value in theta) + ")")
print(f"train:      log_loss={train_loss:.6f} "
      f"accuracy={train_accuracy:.3f} brier={train_brier:.6f}")
print(f"validation: log_loss={validation_loss:.6f} "
      f"accuracy={validation_accuracy:.3f} brier={validation_brier:.6f}")
print(f"directional derivative relative error: {relative_error:.3e}")
```

R:

```
LAMBDA <- 0.1
ITERATIONS <- 60L
CHECKPOINTS <- c(0L, 1L, 2L, 5L, 10L, 20L, 50L, 60L)

raw_observation <- function(i) {
  # Integer recurrences make the fixture identical across languages.
  x1 <- (((37 * i + 11) %% 101) - 50) / 15
  x2 <- (((53 * i + 7) %% 97) - 48) / 15
  noise <- (((29 * i + 3) %% 19) - 9) / 4
  y <- as.numeric(0.3 + 1.2 * x1 - 0.9 * x2 + noise >= 0)
  c(x1, x2, y)
```

```
}

raw <- t(vapply(0:71, raw_observation, numeric(3)))
raw_train <- raw[1:48, , drop = FALSE]
raw_validation <- raw[49:72, , drop = FALSE]
means <- colMeans(raw_train[, 1:2, drop = FALSE])
centred_train <- sweep(raw_train[, 1:2, drop = FALSE], 2, means, "-")
scales <- sqrt(colMeans(centred_train^2))

prepare <- function(rows) {
  features <- sweep(rows[, 1:2, drop = FALSE], 2, means, "-")
  features <- sweep(features, 2, scales, "/")
  list(x = cbind(1, features), y = rows[, 3])
}

train <- prepare(raw_train)
validation <- prepare(raw_validation)
X_TRAIN <- train$x
Y_TRAIN <- train$y
X_VALIDATION <- validation$x
Y_VALIDATION <- validation$y

softplus <- function(score) {
  pmax(score, 0) + log1p(exp(-abs(score)))
}

sigmoid <- function(score) {
  result <- numeric(length(score))
  positive <- score >= 0
  result[positive] <- 1 / (1 + exp(-score[positive]))
  value <- exp(score[!positive])
  result[!positive] <- value / (1 + value)
  result
}

data_loss <- function(theta, x, y) {
  score <- drop(x %*% theta)
  mean(softplus(score) - y * score)
}

objective_gradient <- function(theta, x = X_TRAIN, y = Y_TRAIN) {
  probability <- sigmoid(drop(x %*% theta))
  gradient <- drop(crossprod(x, probability - y)) / length(y) +
    LAMBDA * theta
  penalty <- 0.5 * LAMBDA * sum(theta^2)
  loss <- data_loss(theta, x, y)
  list(objective = loss + penalty, loss = loss, penalty = penalty,
       gradient = gradient)
}

metrics <- function(theta, x, y) {
  probability <- sigmoid(drop(x %*% theta))
  c(log_loss = data_loss(theta, x, y),
    accuracy = mean((probability >= 0.5) == as.logical(y)),
    brier = mean((probability - y)^2))
}

# Since sigma(s)(1-sigma(s)) <= 1/4, this trace bound is a valid global
```

```r
# Lipschitz bound for the penalised gradient. It is conservative but explicit.
smoothness_bound <- sum(X_TRAIN^2) / (4 * nrow(X_TRAIN)) + LAMBDA
step_size <- 1 / smoothness_bound
theta <- c(0, 0, 0)
trace <- vector("list", ITERATIONS + 1L)
for (k in 0:ITERATIONS) {
  state <- objective_gradient(theta)
  trace[[k + 1L]] <- list(k = k, theta = theta,
                          objective = state$objective, loss = state$loss,
                          penalty = state$penalty,
                          gradient_norm = sqrt(sum(state$gradient^2)))
  if (k < ITERATIONS) theta <- theta - step_size * state$gradient
}

# The checks distinguish implementation correctness from held-out behaviour.
values <- vapply(trace, function(row) row$objective, numeric(1))
stopifnot(all(diff(values) <= 1e-14),
          trace[[length(trace)]]$gradient_norm < 1e-8,
          all(is.finite(values)))
probe <- c(0.2, -0.3, 0.4)
direction <- c(0.5, -0.4, sqrt(0.59))
h <- 1e-6
finite_difference <- (objective_gradient(probe + h * direction)$objective -
                      objective_gradient(probe - h * direction)$objective) /
  (2 * h)
analytic <- sum(objective_gradient(probe)$gradient * direction)
relative_error <- abs(finite_difference - analytic) / max(1, abs(analytic))
stopifnot(relative_error < 1e-8)
reversed_result <- objective_gradient(probe,
                                      X_TRAIN[48:1, , drop = FALSE],
                                      Y_TRAIN[48:1])
original_result <- objective_gradient(probe)
stopifnot(abs(reversed_result$objective - original_result$objective) < 1e-15,
          max(abs(reversed_result$gradient - original_result$gradient)) < 1e-15,
          all(is.finite(softplus(c(1000, -1000)))))

cat(sprintf("regularised logistic: train=48 validation=24 lambda=%.3f\n",
            LAMBDA))
cat(sprintf("global smoothness bound=%.6f step=%.6f\n",
            smoothness_bound, step_size))
cat("step     objective      data_loss        penalty     |gradient|       |theta|\n")
for (row in trace) {
  if (row$k %in% CHECKPOINTS) {
    cat(sprintf("%4d  %13.8f  %13.8f  %13.8f  %13.6e  %13.6e\n",
                row$k, row$objective, row$loss, row$penalty,
                row$gradient_norm, sqrt(sum(row$theta^2))))
  }
}

train_metrics <- metrics(theta, X_TRAIN, Y_TRAIN)
validation_metrics <- metrics(theta, X_VALIDATION, Y_VALIDATION)
cat(sprintf("final theta=(%.6f, %.6f, %.6f)\n",
            theta[1], theta[2], theta[3]))
cat(sprintf("train:      log_loss=%.6f accuracy=%.3f brier=%.6f\n",
            train_metrics["log_loss"], train_metrics["accuracy"],
            train_metrics["brier"]))
cat(sprintf("validation: log_loss=%.6f accuracy=%.3f brier=%.6f\n",
            validation_metrics["log_loss"], validation_metrics["accuracy"],
```

```
            validation_metrics["brier"]))
cat(sprintf("directional derivative relative error: %.3e\n",
            relative_error))
```

Both implementations produce this output in the pinned containers:

```
regularised logistic: train=48 validation=24 lambda=0.100
global smoothness bound=0.850000 step=1.176471
step     objective      data_loss        penalty     |gradient|       |theta|
   0     0.69314718     0.69314718     0.00000000   3.630162e-01   0.000000e+00
   1     0.56842899     0.55930921     0.00911978   2.227265e-01   4.270779e-01
   2     0.52050082     0.49675788     0.02374294   1.444013e-01   6.891000e-01
   5     0.48563675     0.42999412     0.05564262   4.891156e-02   1.054918e+00
  10     0.48070108     0.40591585     0.07478523   1.031794e-02   1.222990e+00
  20     0.48045848     0.40009405     0.08036443   5.401137e-04   1.267789e+00
  50     0.48045781     0.39977592     0.08068189   8.330955e-08   1.270290e+00
  60     0.48045781     0.39977587     0.08068193   4.469891e-09   1.270291e+00
final theta=(0.216911, 1.016088, -0.730858)
train:      log_loss=0.399776 accuracy=0.854 brier=0.120839
validation: log_loss=0.404145 accuracy=0.833 brier=0.124090
directional derivative relative error: 1.841e-11
```

The training objective falls monotonically to the printed precision and its gradient norm is below the declared 10^{-8} tolerance. The validation loss is close to, but distinct from, the unpenalised training loss; the training objective is larger because it also includes the penalty. Accuracy and Brier score answer different questions from the smooth objective, and none of the validation quantities enters an update.

The logistic example also shows why existence, uniqueness, and identifiability should be separated. Without regularisation, linearly separable data have an infimum of zero log loss, but no finite coefficient vector attains it: increasing the separating coefficients keeps reducing the loss. An optimiser can therefore produce growing parameter norms and improving training loss without converging to a finite minimiser. Positive ℓ_2 regularisation makes the objective coercive and, when every direction is penalised, strongly convex, so a unique finite minimiser exists.

Uniqueness of parameters is not always required for unique predictions. Collinear features can create multiple coefficient vectors with the same linear score. Neural networks add permutation and rescaling symmetries that make many parameter vectors represent the same function. The optimisation contract should state whether the object of interest is a parameter vector, a prediction function, or merely an objective value. A distance-to-parameter guarantee is meaningful only when the chosen parameterisation identifies a suitable target. Convexity describes the training objective. Regularisation changes that objective and can improve conditioning. Validation estimates predictive behaviour

under a sampling design. A classification threshold converts probabilities to decisions and belongs to the decision rule, not to the smooth training objective.

2.4 Objective, Metric, and Constraint Are Different Roles

The training objective must be computable often and, for gradient methods, differentiable or proximable. The acceptance metric should represent the task. The two need not be identical. Accuracy is piecewise constant in the parameters and gives no useful local gradient, so a smooth surrogate such as log loss is optimised while accuracy, calibration, or a cost-sensitive metric is evaluated separately.

A requirement can enter in three ways. A hard constraint declares some parameter values infeasible. A penalty allows violation but charges for it. A diagnostic measures the property without changing training. These are not interchangeable: a finite penalty does not guarantee feasibility, while a hard constraint can make the numerical problem more difficult or even infeasible.

Before choosing an algorithm, write an optimisation contract containing the exact objective, units and averaging convention, parameter domain, data split, regularisation, constraints, randomness, stopping rule, and acceptance metrics. This contract is the reference against which gradients and solver output can be checked.

Before moving from formulation to geometry, perform a small objective audit:

1. Expand the objective into data, regularisation, and constraint terms, including every normalising constant.
2. State which quantities are sampled and which are computed exactly on every update.
3. Identify coupling across examples, parameters, or time steps that prevents a simple finite-sum interpretation.
4. Check whether a minimiser exists and whether parameter uniqueness matters to the claim you want to make.
5. Test transformations that should leave the objective unchanged, such as permuting examples or repartitioning a deterministic full batch.
6. Record which evaluation metric is deliberately absent from the training objective.

This audit is short, but it prevents a common category error: spending days

tuning an optimiser for an objective that changes with batch size, weighting convention, or data order.

2.5 Worked Formulation Audit: Weighted Logistic Regression

Suppose positive examples receive weight w_+ and negative examples receive weight w_-. One defensible objective is

$$F(\theta) = \frac{1}{W}\sum_{i=1}^{n} w_{y_i}\left[\max(s_i, 0) - y_i s_i + \log(1 + \exp(-|s_i|))\right] + \frac{\lambda}{2}\|\theta\|^2, \qquad W = \sum_{i=1}^{n} w_{y_i}.$$

Its data gradient is divided by W, while the regularisation gradient is $\lambda\theta$. If a mini-batch is sampled uniformly, the weighted batch sum needs a scaling rule whose expectation matches the full weighted average. If examples are sampled with probabilities proportional to their class weights, an uncorrected batch average estimates a different objective. Both procedures may be useful, but they are not two implementations of the same formula.

The audit now has concrete tests. Multiplying both class weights by ten should leave the normalised data term unchanged. Duplicating every observation should also leave it unchanged. Setting $w_+ = w_-$ should recover the ordinary empirical mean. Setting $\lambda = 0$ should remove both the penalty value and its gradient. Reordering examples must not change a deterministic full-batch result. If any test fails, optimiser comparisons are premature.

The acceptance metric remains separate. Class weighting may improve recall for the positive class, but the final threshold still determines the operational trade-off between false positives and false negatives. Selecting that threshold on validation data is an outer decision, not part of solving the differentiable inner objective.

3 Geometry, Derivatives, and Conditioning

An optimiser sees local information about a surface. The gradient supplies a slope, the Hessian supplies local curvature, and constraints limit the available directions. Conditioning determines whether one local scale is representative of the rest.

3.1 Directional Derivatives, Gradients, Jacobians, and Hessians

For a differentiable scalar function $F : \mathbb{R}^d \to \mathbb{R}$, the directional derivative at θ in direction $\mathbf{v}$ is

$$D_{\mathbf{v}}F(\theta) = \lim_{t \to 0} \frac{F(\theta + t\mathbf{v}) - F(\theta)}{t} = \nabla F(\theta)^\top \mathbf{v}.$$

Among unit vectors, $-\nabla F / \|\nabla F\|$ gives the steepest local decrease in the Euclidean norm. That statement depends on the norm and coordinate system.

To see the dependence explicitly, let $\|\cdot\|$ be the norm used to measure a unit step and let $\|\cdot\|_*$ be its dual norm. The best first-order decrease among directions with $\|\mathbf{p}\| \leq 1$ solves

$$\min_{\|\mathbf{p}\| \leq 1} \nabla F(\theta)^\top \mathbf{p} = -\|\nabla F(\theta)\|_*.$$

For the Euclidean norm, the solution is the normalised negative gradient. For a quadratic norm $\|\mathbf{p}\|_{\mathbf{M}} = (\mathbf{p}^\top \mathbf{M} \mathbf{p})^{1/2}$ with $\mathbf{M}$ positive definite, the steepest direction is proportional to $-\mathbf{M}^{-1}\nabla F$. Preconditioned gradient descent is therefore ordinary steepest descent in a different local metric.

This viewpoint unifies several methods without pretending they are identical. Feature standardisation changes the coordinates. A diagonal adaptive method changes a coordinate-wise metric over time. Newton's method uses the Hessian as a local metric when that Hessian is positive definite. Natural-gradient methods use a metric derived from changes in a probability distribution rather than Euclidean changes in parameters. In each case, the algorithm decides which parameter displacement counts as "large".

The metric must still be numerically usable. If $\mathbf{M}$ is nearly singular, the nominally steepest direction can contain enormous components. If it is estimated from noisy gradients, rapid metric changes can destabilise the update. Record both the transformed gradient and the actual parameter step; the latter is what changes the model. Rescaling a parameter changes the Euclidean geometry and can change the path of gradient descent without changing the underlying predictions represented by the model.

For a vector-valued map $r : \mathbb{R}^d \to \mathbb{R}^m$, the Jacobian $\mathbf{J}_r$ collects first derivatives

of each output with respect to each parameter. If $F(\theta) = \frac{1}{2}\|r(\theta)\|^2$, then

$$\nabla F = \mathbf{J}_r^\top r, \qquad \nabla^2 F = \mathbf{J}_r^\top \mathbf{J}_r + \sum_{j=1}^{m} r_j \nabla^2 r_j.$$

The Gauss–Newton method drops the second term. This is accurate near a solution with small residuals or when the residuals are close to linear in the parameters. It is not an exact Newton step in general.

For a twice differentiable scalar objective, the Hessian contains second partial derivatives. A local Taylor model around θ is

$$F(\theta + \mathbf{p}) \approx F(\theta) + \nabla F(\theta)^\top \mathbf{p} + \frac{1}{2}\mathbf{p}^\top \nabla^2 F(\theta)\mathbf{p}.$$

A positive-definite Hessian makes this local quadratic bowl-shaped. An indefinite Hessian has directions of positive and negative curvature, which is the local signature of a saddle. A singular Hessian has at least one locally flat direction.

For a symmetric Hessian, the Rayleigh quotient

$$q(\mathbf{v}) = \frac{\mathbf{v}^\top \nabla^2 F(\theta)\mathbf{v}}{\mathbf{v}^\top \mathbf{v}}$$

is the directional curvature. Its minimum and maximum over non-zero directions are the smallest and largest eigenvalues. A large positive maximum limits a stable first-order step. A very small positive minimum produces a long, weakly curved direction. A negative minimum proves that the local quadratic model has a descent direction not visible from the gradient alone when the gradient is near zero.

The ratio of extreme positive eigenvalues is a useful local condition measure, but it is not always a global condition number. In a non-convex objective, eigenvalues change with the iterate and can cross zero. In an overparameterised model, exact or approximate symmetries can create zero eigenvalues even when predictions are stable. Report the region and object being conditioned: a Hessian at one iterate, a Gauss–Newton matrix, a normal equation, or the complete optimisation problem.

Strong convexity is a global lower-curvature statement on a declared set. Local

positive definiteness near a candidate minimiser is weaker. It can justify a local Newton rate without saying that another basin does not exist. Conversely, a singular Hessian does not by itself prove failure: the objective may be flat along a symmetry direction while still increasing in every direction that changes the represented function.

3.2 Smoothness and Curvature Bounds

An objective has an L-Lipschitz gradient on a region if

$$\|\nabla F(\mathbf{u}) - \nabla F(\mathbf{v})\| \leq L\|\mathbf{u} - \mathbf{v}\|$$

for all points in that region. This smoothness condition bounds how quickly the gradient changes. It yields the upper quadratic model

$$F(\mathbf{v}) \leq F(\mathbf{u}) + \nabla F(\mathbf{u})^\top(\mathbf{v} - \mathbf{u}) + \frac{L}{2}\|\mathbf{v} - \mathbf{u}\|^2.$$

If F is twice differentiable, convex, and its Hessian eigenvalues lie between μ and L, then F is μ-strongly convex and L-smooth. Strong convexity gives a unique minimiser and turns objective or gradient error into distance bounds. Smoothness alone does not imply convexity, and convexity alone does not imply a Lipschitz gradient.

3.3 Recurring Example: A Rotated Ill-Conditioned Quadratic

Let the known minimiser be $\theta^* = (1, -1)^\top$. Define a rotation through 30 degrees,

$$\mathbf{U} = \begin{bmatrix} \cos(\pi/6) & -\sin(\pi/6) \\ \sin(\pi/6) & \cos(\pi/6) \end{bmatrix},$$

and let

$$\mathbf{Q} = \mathbf{U}\begin{bmatrix} 1 & 0 \\ 0 & 100 \end{bmatrix}\mathbf{U}^\top, \qquad F_q(\theta) = \frac{1}{2}(\theta - \theta^*)^\top\mathbf{Q}(\theta - \theta^*).$$

The gradient is $\mathbf{Q}(\theta - \theta^*)$ and the Hessian is the constant matrix $\mathbf{Q}$. Rotation changes the eigenvectors but not the eigenvalues, so $\mu = 1$, $L = 100$, and $\kappa = 100$. Contours are long, narrow ellipses tilted away from the coordinate axes.

With a fixed gradient-descent step $\alpha = 1/L$, the error in an eigenvector direction with eigenvalue ρ is multiplied by $1 - \alpha\rho$. The high-curvature component disappears in one exact arithmetic step, while the low-curvature component is multiplied by 0.99 each time. After 100 iterations, that component is still about $0.99^{100} \approx 0.366$ of its initial size. The objective is simple and convex; the slow progress comes entirely from unequal scales.

A coordinate change $\mathbf{u} = \mathbf{Q}^{1/2}(\theta - \theta^*)$ turns the objective into $\frac{1}{2}\|\mathbf{u}\|^2$. Equivalently, a preconditioned step using $\mathbf{Q}^{-1}$ reaches the minimiser in one iteration. Real problems rarely provide the exact inverse Hessian cheaply, but feature scaling, normalisation, diagonal preconditioners, and curvature approximations pursue the same goal: make progress comparable across directions.

This deterministic experiment uses the recurring quadratic with eigenvalues 1 and 100, a 30-degree rotation, minimiser $(1, -1)$, and condition number 100. It compares the conservative step $1/L$, the fixed step $2/(L + \mu)$ that minimises the worst-case spectral contraction over $[\mu, L]$, Polyak's heavy-ball parameters for that interval, and one exact Newton step. The code prints objective value, gradient norm, and parameter error separately; checks the analytic gradient by a directional finite difference; and asserts monotone objective descent only for the two gradient-descent traces.

Python:

```python
from math import cos, hypot, isfinite, pi, sin, sqrt


MU, L = 1.0, 100.0
ANGLE = pi / 6.0
C, S = cos(ANGLE), sin(ANGLE)
THETA_STAR = (1.0, -1.0)
THETA_0 = (-3.0, 3.0)
CHECKPOINTS = {0, 1, 2, 5, 10, 20, 50, 100}


def rotate_to_eigenbasis(theta):
    """Return R^T(theta - theta_star) for R=[[c,-s],[s,c]]."""
    dx = theta[0] - THETA_STAR[0]
    dy = theta[1] - THETA_STAR[1]
    return (C * dx + S * dy, -S * dx + C * dy)
```

```
def objective(theta):
    z1, z2 = rotate_to_eigenbasis(theta)
    return 0.5 * (MU * z1 * z1 + L * z2 * z2)


def gradient(theta):
    z1, z2 = rotate_to_eigenbasis(theta)
    return (C * MU * z1 - S * L * z2,
            S * MU * z1 + C * L * z2)


def diagnostics(theta):
    value = objective(theta)
    g = gradient(theta)
    error = hypot(theta[0] - THETA_STAR[0], theta[1] - THETA_STAR[1])
    return value, hypot(*g), error


def gradient_descent(step, iterations=100):
    theta = THETA_0
    trace = []
    for k in range(iterations + 1):
        trace.append((k, theta, *diagnostics(theta)))
        if k < iterations:
            g = gradient(theta)
            theta = (theta[0] - step * g[0], theta[1] - step * g[1])
    return trace


def heavy_ball(iterations=100):
    step = 4.0 / (sqrt(L) + sqrt(MU)) ** 2
    momentum = ((sqrt(L) - sqrt(MU)) / (sqrt(L) + sqrt(MU))) ** 2
    theta = THETA_0
    velocity = (0.0, 0.0)
    trace = []
    for k in range(iterations + 1):
        trace.append((k, theta, *diagnostics(theta)))
        if k < iterations:
            g = gradient(theta)
            velocity = (momentum * velocity[0] - step * g[0],
                        momentum * velocity[1] - step * g[1])
            theta = (theta[0] + velocity[0], theta[1] + velocity[1])
    return trace


def newton_step(theta):
    z1, z2 = rotate_to_eigenbasis(theta)
    # Q^{-1} grad(theta) = theta - theta_star for this quadratic.
    correction = (C * z1 - S * z2, S * z1 + C * z2)
    return (theta[0] - correction[0], theta[1] - correction[1])


methods = {
    "GD 1/L": gradient_descent(1.0 / L),
    "GD minimax": gradient_descent(2.0 / (L + MU)),
    "Heavy ball": heavy_ball(),
}
```

```
# Central-difference and convergence checks make the printed trace testable.
direction = (0.6, -0.8)
probe = (0.25, -0.5)
h = 1e-6
fd = (objective((probe[0] + h * direction[0],
                 probe[1] + h * direction[1]))
      - objective((probe[0] - h * direction[0],
                   probe[1] - h * direction[1]))) / (2.0 * h)
g = gradient(probe)
analytic = g[0] * direction[0] + g[1] * direction[1]
assert abs(fd - analytic) <= 1e-7 * max(1.0, abs(analytic))
for name in ("GD 1/L", "GD minimax"):
    values = [row[2] for row in methods[name]]
    assert all(right <= left + 1e-12
               for left, right in zip(values, values[1:]))
newton = newton_step(THETA_0)
assert hypot(newton[0] - THETA_STAR[0],
             newton[1] - THETA_STAR[1]) < 1e-12
assert all(isfinite(item)
           for trace in methods.values() for row in trace for item in row[2:])

print("rotated quadratic: eigenvalues=(1, 100), condition=100")
print("method          k       objective       |gradient|        distance")
for name, trace in methods.items():
    for k, _theta, value, grad_norm, distance in trace:
        if k in CHECKPOINTS:
            print(f"{name:<12} {k:3d}  {value:14.6e}  "
                  f"{grad_norm:14.6e}  {distance:14.6e}")
value, grad_norm, distance = diagnostics(newton)
print(f"{'Newton':<12} {1:3d}  {value:14.6e}  "
      f"{grad_norm:14.6e}  {distance:14.6e}")
print(f"directional derivative relative error: "
      f"{abs(fd - analytic) / abs(analytic):.3e}")
```

R:

```
MU <- 1
L <- 100
ANGLE <- pi / 6
C <- cos(ANGLE)
S <- sin(ANGLE)
THETA_STAR <- c(1, -1)
THETA_0 <- c(-3, 3)
CHECKPOINTS <- c(0L, 1L, 2L, 5L, 10L, 20L, 50L, 100L)

rotate_to_eigenbasis <- function(theta) {
  # Return t(R) %*% (theta - theta_star) for R = [[c,-s],[s,c]].
  dx <- theta[1] - THETA_STAR[1]
  dy <- theta[2] - THETA_STAR[2]
  c(C * dx + S * dy, -S * dx + C * dy)
}

objective <- function(theta) {
  z <- rotate_to_eigenbasis(theta)
  0.5 * (MU * z[1]^2 + L * z[2]^2)
}

gradient <- function(theta) {
  z <- rotate_to_eigenbasis(theta)
  c(C * MU * z[1] - S * L * z[2],
```

```
    S * MU * z[1] + C * L * z[2])
}

diagnostics <- function(theta) {
  g <- gradient(theta)
  c(objective = objective(theta), gradient_norm = sqrt(sum(g^2)),
    distance = sqrt(sum((theta - THETA_STAR)^2)))
}

gradient_descent <- function(step, iterations = 100L) {
  theta <- THETA_0
  trace <- vector("list", iterations + 1L)
  for (k in 0:iterations) {
    trace[[k + 1L]] <- list(k = k, theta = theta,
                            diagnostics = diagnostics(theta))
    if (k < iterations) theta <- theta - step * gradient(theta)
  }
  trace
}

heavy_ball <- function(iterations = 100L) {
  step <- 4 / (sqrt(L) + sqrt(MU))^2
  momentum <- ((sqrt(L) - sqrt(MU)) / (sqrt(L) + sqrt(MU)))^2
  theta <- THETA_0
  velocity <- c(0, 0)
  trace <- vector("list", iterations + 1L)
  for (k in 0:iterations) {
    trace[[k + 1L]] <- list(k = k, theta = theta,
                            diagnostics = diagnostics(theta))
    if (k < iterations) {
      velocity <- momentum * velocity - step * gradient(theta)
      theta <- theta + velocity
    }
  }
  trace
}

newton_step <- function(theta) {
  z <- rotate_to_eigenbasis(theta)
  # solve(Q, gradient(theta)) = theta - theta_star for this quadratic.
  correction <- c(C * z[1] - S * z[2], S * z[1] + C * z[2])
  theta - correction
}

methods <- list("GD 1/L" = gradient_descent(1 / L),
                "GD minimax" = gradient_descent(2 / (L + MU)),
                "Heavy ball" = heavy_ball())

# Central-difference and convergence checks make the printed trace testable.
direction <- c(0.6, -0.8)
probe <- c(0.25, -0.5)
h <- 1e-6
finite_difference <- (objective(probe + h * direction) -
                      objective(probe - h * direction)) / (2 * h)
analytic <- sum(gradient(probe) * direction)
stopifnot(abs(finite_difference - analytic) <=
          1e-7 * max(1, abs(analytic)))
for (name in c("GD 1/L", "GD minimax")) {
```

```
  values <- vapply(methods[[name]],
                   function(row) row$diagnostics["objective"], numeric(1))
  stopifnot(all(diff(values) <= 1e-12))
}
newton <- newton_step(THETA_0)
stopifnot(sqrt(sum((newton - THETA_STAR)^2)) < 1e-12)

cat("rotated quadratic: eigenvalues=(1, 100), condition=100\n")
cat("method          k       objective       |gradient|        distance\n")
for (name in names(methods)) {
  for (row in methods[[name]]) {
    if (row$k %in% CHECKPOINTS) {
      value <- row$diagnostics
      cat(sprintf("%-12s %3d  %14.6e  %14.6e  %14.6e\n", name, row$k,
                  value["objective"], value["gradient_norm"],
                  value["distance"]))
    }
  }
}
value <- diagnostics(newton)
cat(sprintf("%-12s %3d  %14.6e  %14.6e  %14.6e\n", "Newton", 1,
            value["objective"], value["gradient_norm"], value["distance"]))
cat(sprintf("directional derivative relative error: %.3e\n",
            abs(finite_difference - analytic) / abs(analytic)))
```

Both implementations produce this output in the pinned containers:

```
rotated quadratic: eigenvalues=(1, 100), condition=100
method          k       objective       |gradient|        distance
GD 1/L          0    1.493892e+03    5.464121e+02    5.656854e+00
GD 1/L          1    1.050468e+00    1.449461e+00    1.449461e+00
GD 1/L          2    1.029564e+00    1.434966e+00    1.434966e+00
GD 1/L          5    9.693138e-01    1.392346e+00    1.392346e+00
GD 1/L         10    8.766300e-01    1.324107e+00    1.324107e+00
GD 1/L         20    7.170018e-01    1.197499e+00    1.197499e+00
GD 1/L         50    3.923123e-01    8.857904e-01    8.857904e-01
GD 1/L        100    1.435990e-01    5.359085e-01    5.359085e-01
GD minimax      0    1.493892e+03    5.464121e+02    5.656854e+00
GD minimax      1    1.435314e+03    5.355921e+02    5.544837e+00
GD minimax      2    1.379033e+03    5.249863e+02    5.435039e+00
GD minimax      5    1.223087e+03    4.944125e+02    5.118516e+00
GD minimax     10    1.001372e+03    4.473614e+02    4.631410e+00
GD minimax     20    6.712311e+02    3.662661e+02    3.791852e+00
GD minimax     50    2.021628e+02    2.010071e+02    2.080971e+00
GD minimax    100    2.735794e+01    7.394391e+01    7.655209e-01
Heavy ball      0    1.493892e+03    5.464121e+02    5.656854e+00
Heavy ball      1    7.937798e+03    1.259905e+03    1.267833e+01
Heavy ball      2    1.438099e+04    1.695884e+03    1.701141e+01
Heavy ball      5    2.043514e+04    2.021614e+03    2.024210e+01
Heavy ball     10    9.926366e+03    1.408986e+03    1.410078e+01
Heavy ball     20    6.806182e+02    3.689474e+02    3.691512e+00
Heavy ball     50    2.430584e-02    2.204796e+00    2.205750e-02
Heavy ball    100    1.853594e-10    1.925398e-04    1.926159e-06
Newton          1    0.000000e+00    0.000000e+00    0.000000e+00
directional derivative relative error: 5.706e-11
```

The first $1/L$ step removes the high-curvature component almost completely, after which progress is governed by the slow eigenvalue. The minimax fixed step has the best worst-direction contraction factor, but it does not dominate every

finite trace: for this start it retains a large high-curvature component through the printed horizon. Heavy ball initially overshoots and is not monotone, then overtakes both scalar gradient-descent runs. Newton uses the exact curvature and reaches the known minimiser in one step. These are iteration comparisons on a two-dimensional quadratic, not wall-clock benchmarks.

Large models rarely permit a materialised $d \times d$ Hessian. They can still expose a Hessian-vector product

$$\mathbf{v} \longmapsto \nabla^2 F(\theta)\mathbf{v}$$

at a cost comparable to a small number of gradient evaluations through automatic differentiation. Krylov methods can use this operation to solve Newton systems approximately, estimate extreme eigenvalues, or test curvature along proposed directions. A finite-difference cross-check is

$$\nabla^2 F(\theta)\mathbf{v} \approx \frac{\nabla F(\theta + h\mathbf{v}) - \nabla F(\theta - h\mathbf{v})}{2h}.$$

As with a gradient check, use several h values and a deterministic computation. Agreement for one vector tests one directional action, not the whole Hessian.

Three cheap probes often reveal a scale problem before a sophisticated method is needed. First, compare feature or activation scales and the corresponding gradient scales. Second, measure the ratio $\|\mathbf{p}_k\|/(\|\theta_k\| + \delta)$ for each parameter block, not only for the whole vector. Third, evaluate directional curvature $\mathbf{p}_k^\top \nabla^2 F \mathbf{p}_k / \|\mathbf{p}_k\|^2$ along the actual update. If one block moves by a large fraction of its parameter norm while another is effectively frozen, one global learning rate is solving two different scale problems.

Scaling can improve optimisation while changing statistical meaning. Standardising a feature and transforming its coefficient back preserves an unregularised linear predictor, but an isotropic penalty in the transformed coordinates is not generally isotropic in the original units. State whether a rescaling is a pure reparameterisation or part of the model definition.

3.4 Derivative Checks and Numerical Limits

Automatic differentiation applies the chain rule to program operations; it is neither symbolic algebra nor finite differencing. It can produce derivatives of the implemented computation accurately up to floating-point effects, but it cannot tell you that the implemented loss is the wrong loss, that a tensor was broadcast along the wrong axis, or that data leakage entered the computation.

A central finite-difference check for component j is

$$\frac{F(\theta + h\mathbf{e}_j) - F(\theta - h\mathbf{e}_j)}{2h}.$$

Truncation error dominates when h is too large, while cancellation and round-off dominate when h is too small. Check several logarithmically spaced values and compare directional derivatives rather than every component when d is large. Run the check in a deterministic, smooth regime: disable stochastic layers, fix the batch, and avoid points exactly at a non-differentiable kink.

For the quadratic example, the gradient check should approach the analytic directional derivative and then deteriorate at very small h. For logistic regression, compare the analytic gradient against finite differences before trusting an optimiser trace. A passed check supports the derivative implementation at the tested points; it does not prove the whole training system correct.

3.5 Automatic Differentiation for Optimisation

Automatic differentiation (AD) is the bridge between the mathematical derivatives in this chapter and the derivatives supplied to an optimiser. The Numerical Analysis primer's automatic-differentiation chapter develops dual numbers, computation tapes, and the mechanics of forward and reverse mode. Here the practical question is different: which derivative object does an optimisation method need, how should it be computed, and what evidence supports the claim that it is the derivative of the intended objective?

3.5.1 The Derivative Object the Solver Needs

Let $r : \mathbb{R}^d \to \mathbb{R}^m$ be a vector-valued computation with Jacobian $\mathbf{J}_r \in \mathbb{R}^{m \times d}$. A forward-mode calculation seeded by a tangent $\mathbf{v} \in \mathbb{R}^d$ returns the Jacobian-

vector product (JVP)

$$\mathbf{J}_r(\theta)\mathbf{v} \in \mathbb{R}^m.$$

A reverse-mode calculation seeded by a cotangent $\mathbf{u} \in \mathbb{R}^m$ returns the vector-Jacobian product (VJP)

$$\mathbf{J}_r(\theta)^\top \mathbf{u} \in \mathbb{R}^d.$$

Neither operation requires storing the complete Jacobian. For a scalar objective F, reverse mode with output seed 1 gives ∇F. This is why reverse mode is usually natural for a loss with millions of parameters and one scalar output. Forward mode is attractive when there are few input directions or many outputs. A full Jacobian needs roughly one seed per input direction in forward mode or one seed per output direction in reverse mode; memory and primitive implementation details also affect the actual cost.

These products map directly to optimisation methods. A Gauss–Newton action is

$$\mathbf{J}_r^\top(\mathbf{J}_r\mathbf{v}),$$

so a JVP followed by a VJP can apply it without materialising $\mathbf{J}_r$. For a scalar objective, an HVP is

$$\nabla^2 F(\theta)\mathbf{v} = D_{\mathbf{v}}(\nabla F)(\theta).$$

Forward-over-reverse computes this by applying a JVP to the gradient function. This is useful for Krylov and truncated-Newton methods, curvature probes, and some hypergradient calculations. An HVP is an action of the Hessian, not the Hessian itself; forming a dense matrix merely to multiply by one vector wastes memory on a large problem.

The familiar backpropagation pass is reverse-mode accumulation through a model and its scalar loss. The optimiser receives the gradient of the complete implemented objective, including reductions, masks, regularisation, and

transformations between parameters and predictions. A gradient of the network output before the loss reduction is not necessarily the gradient that the optimiser should use.

3.5.2 From a Written Loss to an Executable Objective

Consider the regularised logistic objective already used in this primer:

$$F(\theta) = \frac{1}{n}\sum_{i=1}^{n}\left[\text{softplus}(\mathbf{x}_i^\top\theta) - y_i\mathbf{x}_i^\top\theta\right] + \frac{\lambda}{2}\|\theta\|^2.$$

Its executable graph contains a matrix product, a stable elementwise loss, a mean reduction, and a penalty. Omitting the mean multiplies the data gradient by n while leaving the penalty unchanged. Penalising an intercept that the mathematical statement excludes changes the problem. AD can differentiate either implementation accurately; it cannot decide which one was intended.

The same warning applies to graph boundaries. Converting a differentiable value to an ordinary number, detaching it, passing through an unsupported external operation, or updating a parameter under a no-gradient context can remove a path that appears in the written derivation. Control flow can select a different executed branch at another parameter value. A successful derivative call therefore establishes a derivative of the executed program, subject to the rules supplied for its operations, not a proof that the program expresses the stated model.

3.5.3 Memory, State, and Non-Smooth Operations

Reverse mode normally retains or reconstructs intermediate values needed by the backward pass. Deep models, long unrolled optimisers, and large batches can therefore be limited by activation memory rather than arithmetic. Checkpointing or rematerialisation trades additional forward computation for lower memory. Record such choices when they affect feasible batch size or measured cost.

Stochastic and stateful computations need an explicit boundary. With a fixed batch and random state, AD differentiates the realised computation. Across random batches it produces a sequence of minibatch gradients; whether their

expectation estimates the full objective is a property of sampling and reduction, not of AD. Batch-dependent normalisation, running statistics, mutable buffers, training/evaluation mode, and unrolled optimiser state can all change the function being differentiated. Freeze or record those quantities during derivative checks.

At a non-smooth operation such as $\max(0, s)$, the classical derivative may not exist at the kink. A framework must choose a convention there, while clipping, sorting, top-k selection, and hard thresholds can make nearby branch changes invisible to a local derivative. For an ℓ_1 penalty, a proximal method may express the intended problem more faithfully than treating one selected subgradient as an ordinary smooth gradient. Check the solver's regularity assumptions against the operations in the graph.

3.5.4 Worked Derivative Products

The following small example uses the same four-point logistic data in three ecosystems. The Python and R blocks use reverse-mode AD for the gradient and a mixed-mode or nested construction for the HVP. The Julia block contrasts `ForwardDiff.jl`'s forward-mode gradient with `Zygote.jl`'s reverse-mode gradient and HVP. These package examples demonstrate derivative objects; they are not performance benchmarks.

Python:

```python
import torch
from torch.func import grad, jvp, vjp

torch.set_default_dtype(torch.float64)
X = torch.tensor([[1., 0.], [1., 1.], [1., 2.], [1., -1.]])
y = torch.tensor([0., 1., 1., 0.])
lam = 0.1
theta = torch.tensor([0.2, -0.3])
v = torch.tensor([0.4, -0.2])

def logits(t):
    return X @ t

def objective(t):
    s = logits(t)
    return (torch.nn.functional.softplus(s) - y * s).mean() \
        + 0.5 * lam * (t @ t)

gradient = grad(objective)
g = gradient(theta)
_, hv = jvp(gradient, (theta,), (v,))
_, pullback = vjp(logits, theta)
_, jvp_logits = jvp(logits, (theta,), (v,))
```

```
weights = torch.sigmoid(logits(theta)) * (1 - torch.sigmoid(logits(theta)))
expected_hv = X.T @ (weights * (X @ v)) / len(y) + lam * v
assert torch.allclose(hv, expected_hv, atol=1e-12, rtol=1e-12)
assert torch.allclose(jvp_logits, X @ v, atol=1e-12, rtol=1e-12)
print("gradient:", g, "HVP:", hv)
```

R:

```
library(torch)

dtype <- torch_float64()
X <- torch_tensor(rbind(c(1, 0), c(1, 1), c(1, 2), c(1, -1)), dtype = dtype)
y <- torch_tensor(c(0, 1, 1, 0), dtype = dtype)
lam <- 0.1
theta <- torch_tensor(c(0.2, -0.3), dtype = dtype, requires_grad = TRUE)
v <- torch_tensor(c(0.4, -0.2), dtype = dtype)

objective <- function(t) {
  s <- torch_matmul(X, t)
  (nnf_softplus(s) - y * s)$mean() + 0.5 * lam * (t * t)$sum()
}

loss <- objective(theta)
g <- autograd_grad(loss, list(theta), create_graph = TRUE)[[1]]
hv <- autograd_grad((g * v)$sum(), list(theta))[[1]]
print(list(gradient = as_array(g$detach()), hvp = as_array(hv$detach())))
```

Julia:

```
using ForwardDiff
using Zygote
using LinearAlgebra
using Statistics

X = [1.0 0.0; 1.0 1.0; 1.0 2.0; 1.0 -1.0]
y = [0.0, 1.0, 1.0, 0.0]
λ = 0.1
θ = [0.2, -0.3]
v = [0.4, -0.2]

softplus(s) = max(s, 0) + log1p(exp(-abs(s)))
objective(t) = mean(softplus.(X * t) - y .* (X * t)) + λ / 2 * dot(t, t)
logits(t) = X * t

forward_gradient = ForwardDiff.gradient(objective, θ)
reverse_gradient = Zygote.gradient(objective, θ)[1]
jvp_logits = ForwardDiff.derivative(a -> logits(θ .+ a .* v), 0.0)
hvp = Zygote.hessian(objective, θ) * v  # dense reference for this tiny case

@assert isapprox(forward_gradient, reverse_gradient; atol = 1e-10)
@assert isapprox(jvp_logits, X * v; atol = 1e-10)
println("gradient = ", forward_gradient, ", HVP = ", hvp)
```

The Python JVP pushes one parameter perturbation through the logit map; its VJP pullback maps an output weighting back to parameter space. The R example uses `create_graph = TRUE` so the first derivative remains available to the second derivative calculation. The Julia example uses different AD modes

deliberately: `ForwardDiff.gradient` and the scalar-seeded JVP use forward sensitivities, while Zygote constructs the reverse-mode gradient. Its dense Hessian is only a reference for this two-parameter fixture; larger problems should use matrix-free products and choose modes according to dimensions, memory, and the solver's request.

3.5.5 Validate AD Independently

Begin with a deterministic fixture and write down the exact scalar objective, reduction, parameter mask, regularisation coefficient, and computation mode. Check value-level invariants first, such as permutation invariance for an unweighted full-batch mean. Then compare the AD directional derivative $\nabla F(\theta)^\top \mathbf{v}$ with a central difference of objective values and compare an HVP with a central difference of gradients. Use several step sizes: one large step can cross significant curvature or a branch, while one tiny step can be dominated by cancellation.

For a vector-valued map, the identity

$$\mathbf{u}^\top(\mathbf{J}_r\mathbf{v}) = (\mathbf{J}_r^\top\mathbf{u})^\top\mathbf{v}$$

checks a JVP and VJP pair without constructing the Jacobian. Probe multiple points and directions, including broadcasting, masks, regularisation, and parameter blocks that should receive no gradient. Freeze batches, random masks, and stateful layers during these tests.

A passing AD check is local evidence about a derivative calculation. It does not establish that the intended objective was encoded, that a custom derivative rule is correct everywhere, that the computation is numerically stable, or that the optimiser has converged. Record the objective definition, derivative test, solver settings, and termination evidence together.

For API details, see the official JAX differentiation guide, R `torch` documentation for autograd and `autograd_grad`, and Julia documentation for ForwardDiff.jl and Zygote higher-order derivatives. Package APIs were checked against these official pages on 18 October 2026; exact installed versions and execution remain unverified in this workspace.

3.6 Conditioning Beyond One Number

The ratio L/μ summarises the worst curvature scales for a strongly convex smooth problem, but it can be pessimistic. An initial error may have almost no component in the most difficult eigendirection. Eigenvalues may form clusters that an iterative method handles quickly even when the extreme ratio is large. Conversely, a moderate global ratio can hide a parameter block whose scale is poorly matched to the rest of the implementation.

For a quadratic with eigendecomposition $\mathbf{Q} = \mathbf{U}\Lambda\mathbf{U}^\top$, write the error as $\mathbf{e}_k = \sum_j a_{k,j}\mathbf{u}_j$. Gradient descent with step α evolves each component independently:

$$a_{k+1,j} = (1 - \alpha\lambda_j)a_{k,j}.$$

Components with $\alpha\lambda_j$ near one disappear quickly; components with $\alpha\lambda_j$ near zero barely move; components with $\alpha\lambda_j > 2$ are unstable. A trace that initially falls rapidly and then develops a long tail often reflects removal of high-curvature components followed by slow motion in weakly curved directions.

Preconditioning replaces the update by $-\alpha\mathbf{P}\nabla F$ for a positive-definite matrix $\mathbf{P}$. On the quadratic, the transformed dynamics depend on the spectrum of $\mathbf{P}^{1/2}\mathbf{Q}\mathbf{P}^{1/2}$. The goal is not necessarily to approximate $\mathbf{Q}^{-1}$ accurately in every entry; it is to cluster the effective eigenvalues enough that one step size works across important directions. A diagonal preconditioner can repair unit differences but cannot remove arbitrary rotated coupling.

Estimate conditioning with several pieces of evidence: extreme directional curvatures, convergence slopes on a log residual plot, block-level update ratios, and sensitivity to rescaling. Name the point, matrix approximation, and subspace used for any reported Hessian condition estimate.

4 Problem Classes and Convergence Claims

Method names are less informative than problem classes. A guarantee has four parts: assumptions, a quantity that decreases, a rate or limiting claim, and a unit of work. Removing any part can turn a theorem into a misleading slogan.

4.1 Classify Before You Optimise

An unconstrained smooth problem has a differentiable objective and no explicit feasible set. A constrained problem restricts θ to $\mathcal{C}$ or imposes equalities and inequalities. A composite problem has a smooth part plus a simple non-smooth part, such as an ℓ_1 penalty. A stochastic problem exposes only noisy objective or gradient estimates. These categories overlap.

Convexity means that the line segment between two points lies above the graph's supporting chord:

$$F(t\mathbf{u} + (1-t)\mathbf{v}) \leq tF(\mathbf{u}) + (1-t)F(\mathbf{v})$$

for $t \in [0,1]$. For differentiable F, convexity is equivalent to the global supporting-hyperplane inequality

$$F(\mathbf{v}) \geq F(\mathbf{u}) + \nabla F(\mathbf{u})^\top(\mathbf{v}-\mathbf{u})$$

for every $\mathbf{u}$ and $\mathbf{v}$ in a convex domain. The gradient is not merely a local slope: its affine approximation is a global lower bound. Setting $\nabla F(\theta^*) = \mathbf{0}$ therefore proves that no other point has a smaller value.

For a non-differentiable convex function, replace the gradient by any subgradient $\mathbf{g} \in \partial F(\mathbf{u})$. The same supporting inequality defines the subdifferential. The condition $\mathbf{0} \in \partial F(\theta^*)$ is then sufficient for global optimality. This is why a kink does not invalidate convex optimisation; it changes the stationarity object.

Convexity is preserved by several operations that appear in machine learning: non-negative weighted sums, affine composition, pointwise maxima, and partial minimisation under suitable conditions. It is not preserved by arbitrary composition. A convex loss applied to a linear score can remain convex in the coefficients, while the same loss applied to a multilayer network is generally non-convex in its weights.

Every local minimiser of a convex objective over a convex feasible set is global. For a differentiable convex objective, $\nabla F(\theta^*) = \mathbf{0}$ is sufficient for global optimality. In a non-convex problem, the same equation identifies a stationary point that might be a local minimum, local maximum, or saddle.

Strong convexity is stronger than uniqueness. A strictly convex objective can have a unique minimiser without a uniform curvature lower bound. Strong convexity supplies quantitative inequalities used in linear convergence rates. A regularised logistic objective with $\lambda > 0$ is a central example when all parameter directions are penalised.

A differentiable function is μ-strongly convex when

$$F(\mathbf{v}) \geq F(\mathbf{u}) + \nabla F(\mathbf{u})^\top (\mathbf{v} - \mathbf{u}) + \frac{\mu}{2}\|\mathbf{v} - \mathbf{u}\|^2.$$

At a minimiser, this gives quadratic growth:

$$F(\theta) - F^* \geq \frac{\mu}{2}\|\theta - \theta^*\|^2.$$

It supplies uniqueness and lets objective error control parameter error. The reverse implication does not hold without extra conditions: a function can grow quadratically away from its solution set without being strongly convex everywhere.

Another useful condition is the Polyak–Łojasiewicz inequality,

$$\frac{1}{2}\|\nabla F(\theta)\|^2 \geq \mu_{\mathrm{PL}}\big(F(\theta) - F^*\big).$$

Strong convexity implies this inequality for a smooth differentiable objective, but the inequality can hold for some non-convex functions. Combined with a sufficient-decrease step, it yields a linear objective-rate bound without claiming that the parameterisation is convex or that the minimiser is unique. Name the condition actually used; do not call every linear rate a consequence of strong convexity.

These distinctions matter in overparameterised models. Distance to one selected parameter vector may be meaningless when a whole set of vectors represents the same solution. Objective gap or distance to the solution set can still be meaningful. A theorem's error measure should match the identifiability of the model.

The standard gradient-descent rates come from combining a small number of inequalities. For an L-smooth objective and the step $\theta_{k+1} = \theta_k - L^{-1}\nabla F(\theta_k)$,

the descent lemma gives

$$F(\theta_{k+1}) \leq F(\theta_k) - \frac{1}{2L}\|\nabla F(\theta_k)\|^2.$$

If F is convex, compare the gradient with the vector from the current iterate to a minimiser and telescope a distance inequality. One obtains, for a representative fixed step,

$$F(\theta_k) - F^* \leq \frac{L\|\theta_0 - \theta^*\|^2}{2k}.$$

The $O(1/k)$ notation suppresses the initial distance and smoothness constant. Those constants can dominate at the iteration counts that matter.

If the Polyak–Łojasiewicz inequality also holds, substitute it into the sufficient-decrease inequality:

$$F(\theta_{k+1}) - F^* \leq \left(1 - \frac{\mu_{\mathrm{PL}}}{L}\right)(F(\theta_k) - F^*).$$

This is a linear, or geometric, rate. "Linear" describes contraction of the error, not a straight objective curve and not linear wall-clock scaling.

For a smooth non-convex objective bounded below, summing the descent inequality yields

$$\min_{0 \leq j < k} \|\nabla F(\theta_j)\|^2 \leq \frac{2L\big(F(\theta_0) - F_{\mathrm{low}}\big)}{k},$$

where F_{low} is any valid lower bound. The conclusion is about first-order stationarity of at least one iterate. It does not identify which iterate in advance, prove that the last iterate is best, or distinguish a local minimum from a saddle.

4.2 What Common Rates Actually Say

The following table gives representative deterministic results. Constants, norms, and admissible step sizes vary by theorem, so each row is a pattern rather than a substitute for a method-specific proof.

Problem and Method	Assumptions	Representative Guarantee	What the Guarantee Does Not Say
Convex, L-smooth; gradient descent	A minimiser exists; exact gradients; suitable fixed step	$F(\theta_k) - F^* = O(1/k)$	Parameters need not be close when minimisers are non-unique
μ-strongly convex and L-smooth; gradient descent	Exact gradients; suitable step	Objective and distance errors contract geometrically at a condition-dependent rate	Wall-clock speed can still be poor; an inaccurate gradient can create a floor
Convex, bounded subgradients; subgradient method	Bounded distance to a minimiser; diminishing step or averaging	Best or averaged objective error is often $O(1/\sqrt{k})$	The last iterate need not decrease monotonically
Smooth, possibly non-convex; gradient descent	Objective bounded below; L-Lipschitz gradient; suitable step	The smallest squared gradient norm among the first k iterates is $O(1/k)$	No global-minimum guarantee
Strongly convex stochastic approximation	Unbiased gradient estimates with controlled variance; diminishing steps	Expected error can decay as $O(1/k)$ under standard schedules	Constant steps generally leave a noise neighbourhood
Local Newton method	Twice differentiable; nonsingular Hessian near a suitable minimiser; start close enough	Quadratic local convergence	Global convergence and positive-descent steps require safeguards

Rate statements also differ in their quantifiers. A deterministic statement bounds every run satisfying the assumptions. An expectation bound averages over declared algorithmic randomness and can hide a broad outcome distribution. A high-probability bound gives a failure probability. Almost-sure convergence concerns the limiting event over an infinite sequence. These claims are not interchangeable, and none alone reports variability at the finite budget used in an experiment.

An asymptotic rate describes behaviour as k grows. An iteration-complexity result instead asks how much work is sufficient to reach a tolerance ε. For example, an $O(1/k)$ stationarity bound corresponds to $O(1/\varepsilon)$ iterations for a squared-gradient tolerance, while a geometric contraction gives a logarithmic dependence on $1/\varepsilon$. Constants, setup costs, and the cost of one oracle call remain essential to a method comparison.

Real gradients can be inexact even when they are not stochastic. A truncated inner solve, reduced-precision arithmetic, stale distributed update, approximate adjoint, gradient clipping rule, or biased compression operator produces a gradient error $\mathbf{e}_k$:

$$\mathbf{g}_k = \nabla F(\theta_k) + \mathbf{e}_k.$$

Convergence may survive if errors shrink, remain summable, or are controlled relative to the true gradient. A persistent absolute error commonly creates an accuracy floor. A convergence claim for exact gradients does not automatically apply because the approximation is “small”; the theorem needs an error model.

Iteration complexity counts iterations, gradient calls, Hessian-vector products, or another declared oracle. It does not count data loading, communication, compilation, or parallel efficiency. A method with a better iteration rate can lose in wall-clock time if each iteration is much more expensive.

4.3 Stopping Criteria Are Claims About Evidence

A solver needs a stopping rule before it runs. Common criteria include a small gradient norm, a small projected-gradient or proximal-gradient mapping, a small constraint residual, a small duality gap, a small relative objective change, or exhaustion of a resource budget. These quantities answer different questions.

For an unconstrained differentiable objective, a small gradient is a first-order stationarity certificate. Under μ-strong convexity, it also bounds distance to the minimiser because

$$\|\theta - \theta^*\| \leq \frac{1}{\mu}\|\nabla F(\theta)\|.$$

Without strong convexity, the same gradient norm may coexist with a flat plateau far from a useful solution. A small change in objective can mean convergence, but it can also mean a tiny learning rate, numerical underflow, or motion along a flat direction. A maximum-iteration stop is a budget statement, not a convergence statement.

Stochastic objectives require smoothing or statistical interpretation. A single mini-batch gradient norm is noisy. Estimate it on a fixed diagnostic batch, report variability, or use a moving average with a declared window. Keep the training stopping rule distinct from early stopping selected by validation performance; the latter is a model-selection and regularisation decision.

4.4 A Guarantee Checklist

Before repeating a convergence claim, identify the answers to these questions:

1. Is the objective convex, strongly convex, smooth, non-smooth, or non-convex?
2. Are gradients exact, unbiased with bounded variance, biased, clipped, quantised, or stale?
3. Is the guarantee global or only local, and what initialisation region is required?
4. What step-size, line-search, or trust-region conditions are assumed?
5. What quantity converges: objective gap, iterate distance, gradient norm, regret, feasibility, or duality gap?
6. Is the statement in expectation, with high probability, almost surely, or deterministically?
7. What is one unit of work, and what memory or communication does it require?

If a training run violates an assumption, the theorem has not been disproved; it has become inapplicable. The right response is to choose a result matching the

actual algorithm or weaken the claim.

Add three implementation questions to the checklist:

8. Does the implementation compute the oracle assumed by the theorem, including the same averaging, regularisation, projection, and update order?
9. Are constants such as L, μ, a variance bound, or a constraint qualification global, local, estimated, or merely hypothesised?
10. Does finite precision, an inner stopping tolerance, or distributed staleness create a residual floor above the requested outer tolerance?

A useful reporting template is: "Under assumptions A, B, and C, the implemented quantity Q is bounded in sense S after work unit W. In this experiment, A was checked by X, B is plausible because Y, and C was not verified." This separates theorem, evidence, and unresolved assumption.

4.5 Reading a Convergence Theorem as an Engineering Contract

Consider the statement that gradient descent converges linearly on an L-smooth, μ-strongly convex objective with step $1/L$. To use it, identify six objects: the exact objective, the region on which the constants hold, the computed gradient, the update equation, the error quantity, and the work unit. Changing any one can invalidate the quoted result without making the implementation defective.

The constants need not be known to run the algorithm, but they still govern the claim. Backtracking can replace prior knowledge of L by testing sufficient decrease. It does not establish strong convexity or identify μ. Adding an ℓ_2 penalty can supply a lower curvature bound when every direction is penalised, but leaving an intercept or embedding block unpenalised requires a separate argument.

Local assumptions require a basin statement. Newton's quadratic rate is usually conditional on starting sufficiently close to a suitable minimiser and solving the Newton system accurately enough. A globalised method may guarantee that limit points are stationary while only entering the fast local regime later. Report the global progress result and local rate separately.

Stochastic assumptions require a history $\mathcal{F}_k$ available before drawing the next estimate. Conditional unbiasedness means

$$\mathbb{E}[\mathbf{g}_k \mid \mathcal{F}_k] = \nabla F(\theta_k).$$

Reusing data adaptively, maintaining stateful batch statistics, or selecting examples from current losses can break this condition. An unconditional average computed afterwards is not an adequate substitute.

Finally, distinguish convergence of values, iterates, and residuals. Objective values can converge while iterates move along a flat solution set. Iterates can converge to a stationary point with poor objective value in a non-convex problem. A residual can reach a finite-precision floor while the exact-arithmetic sequence would continue. State exactly which event the evidence supports.

5 First-Order Methods

First-order methods build steps from gradients or subgradients. Their low memory cost and access to mini-batch estimates make them the default at machine-learning scale. The price is sensitivity to scaling and learning-rate choices, especially when curvature varies strongly by direction.

5.1 Gradient Descent and Line Search

Gradient descent uses the update

$$\theta_{k+1} = \theta_k - \alpha_k \nabla F(\theta_k).$$

For an L-smooth objective, any fixed step $0 < \alpha < 2/L$ decreases the objective unless the gradient is zero. The conservative choice $\alpha = 1/L$ gives

$$F(\theta_{k+1}) \leq F(\theta_k) - \frac{1}{2L}\|\nabla F(\theta_k)\|^2.$$

If the objective is also μ-strongly convex, the error contracts at a linear, or geometric, rate controlled by $\kappa = L/\mu$. This is a global guarantee for the stated class. It does not apply when the gradient is stochastic, the objective changes during training, or the selected step exceeds the stability range.

A line search chooses α_k from information along the proposed direction. Backtracking begins with a trial step and shrinks it until a sufficient-decrease condition holds. This avoids knowing a global L, but each rejected trial costs an objective evaluation. A Wolfe line search adds a curvature condition and is especially useful with quasi-Newton methods. Line search assumes repeated evaluations are meaningful; a fresh noisy mini-batch at every trial can make acceptance erratic.

For the rotated quadratic, the fixed step $1/100$ is stable but slow in the low-curvature direction. The best constant step for a quadratic with eigenvalues in $[\mu, L]$ is $2/(L+\mu)$, which minimises the worst contraction factor. It still retains a factor of $(\kappa-1)/(\kappa+1)$, about 0.9802 here. A line search can reduce immediate objective value, but it cannot erase ill-conditioning: successive steepest-descent directions remain poorly aligned with the long valley.

Gradient descent fails visibly when the step is too large: objective values oscillate or diverge, parameter norms grow, and floating-point values can become non-finite. With a very small step, it fails quietly by making negligible progress. Always compare the update norm with the parameter scale and the gradient norm with the expected local curvature.

5.2 Stochastic and Mini-Batch Gradients

For a data average, a mini-batch B_k gives the estimator

$$\mathbf{g}_k = \frac{1}{|B_k|} \sum_{i \in B_k} \nabla \ell(\theta_k; z_i) + \lambda \nabla R(\theta_k).$$

If examples are sampled uniformly and the regulariser is included exactly, then $\mathbb{E}[\mathbf{g}_k \mid \theta_k] = \nabla F(\theta_k)$. Sampling without replacement within an epoch creates dependence between updates, although it often works well in practice. Weighted data, class-balanced batches, clipping, and data-dependent sampling can change the expectation; document the estimator actually used.

Stochastic gradient descent (SGD) replaces the exact gradient in the update with $\mathbf{g}_k$. Under standard convex assumptions, diminishing steps can converge to the optimum in expectation or almost surely, depending on the theorem. With a constant step and persistent gradient variance, iterates generally fluctuate in

a neighbourhood whose scale depends on the step, curvature, and noise. Averaging iterates or reducing the step can shrink that neighbourhood.

Suppose the estimator is conditionally unbiased and has bounded conditional variance:

$$\mathbb{E}[\mathbf{g}_k \mid \theta_k] = \nabla F(\theta_k), \qquad \mathbb{E}\big[\|\mathbf{g}_k - \nabla F(\theta_k)\|^2 \mid \theta_k\big] \le \sigma^2.$$

Smoothness then gives a representative one-step bound for SGD:

$$\mathbb{E}[F(\theta_{k+1}) \mid \theta_k] \le F(\theta_k) - \alpha_k \left(1 - \frac{L\alpha_k}{2}\right) \|\nabla F(\theta_k)\|^2 + \frac{L\alpha_k^2}{2}\sigma^2.$$

The useful descent term is proportional to α_k, while the variance cost is proportional to α_k^2. A constant step can make rapid early progress but eventually balances against noise. A diminishing step suppresses the variance term but also slows movement. This equation explains the noise floor more precisely than the slogan that SGD "converges approximately".

Classical stochastic approximation often assumes

$$\sum_{k=0}^{\infty} \alpha_k = \infty, \qquad \sum_{k=0}^{\infty} \alpha_k^2 < \infty.$$

The first condition prevents the algorithm from stopping too early; the second limits accumulated variance. A schedule such as $\alpha_k = a/(b+k)$ can satisfy both. A finite training run need not use that schedule, but then it should not borrow a theorem that requires it.

The variance is rarely a single constant in practice. It changes with the iterate, batch construction, augmentation, class mix, and model state. Estimate it directly on selected checkpoints by drawing several independent batches and measuring the dispersion of gradients or directional projections. The full covariance is too large to store, but block norms and cosine similarities can distinguish a high-variance estimator from consistently biased directions.

Mini-batching trades variance against cost and hardware utilisation. Doubling batch size does not automatically halve wall-clock time or gradient variance, because examples can be correlated and hardware throughput is not linear. Meas-

ure examples per second, update latency, and validation behaviour instead of treating batch size as a purely mathematical parameter. If batch size changes, a learning-rate scaling rule is a hypothesis to test, not an identity.

For regularised logistic regression, each per-example data gradient is $(p_i - y_i)\mathbf{x}_i$. Its magnitude depends on feature scale. Standardising continuous features can improve conditioning and make one global learning rate more meaningful. The penalty gradient $\lambda\theta$ is not sampling noise and should not be multiplied by an accidental batch-dependent factor.

For uniform sampling without replacement from a finite data set, the variance of a batch mean contains a finite-population correction. If per-example gradients have covariance Σ at the current iterate, a simple random batch of size b has covariance proportional to

$$\frac{1}{b}\left(1 - \frac{b-1}{n-1}\right)\Sigma.$$

The familiar $1/b$ reduction is therefore an approximation when $b \ll n$, not a universal identity. Correlated examples, grouped sampling, sequence windows, and augmentation further change the variance.

Iterate averaging is a separate operation from gradient averaging. Polyak–Ruppert averaging forms

$$\bar{\theta}_K = \frac{1}{K - K_0} \sum_{k=K_0+1}^{K} \theta_k$$

after a burn-in index K_0. Under suitable strongly convex stochastic-approximation assumptions, averaging can attain favourable asymptotic variance and reduce oscillation. Averaging parameters in a model with symmetries can be dangerous, however: two functionally similar neural-network solutions can have permuted hidden units whose arithmetic average represents a poor model. Validate the averaged model rather than assuming that parameter averaging preserves predictions.

Finite-sum objectives permit a middle ground between full gradients and permanently noisy SGD. Variance-reduced methods combine a cheap component gradient with stored or periodically refreshed information. A representative

SVRG estimator uses a reference point $\tilde{\theta}$ and its full gradient:

$$\mathbf{g}_k = \nabla f_{i_k}(\theta_k) - \nabla f_{i_k}(\tilde{\theta}) + \nabla F(\tilde{\theta}).$$

The estimator is unbiased under uniform sampling, and its variance shrinks as the current iterate approaches the reference point. SAGA uses a table of stored component gradients instead of periodic full-gradient snapshots. SARAH-like estimators update a recursive gradient estimate and can be biased conditionally while retaining useful convergence properties under their own analyses.

These methods can recover linear convergence on smooth strongly convex finite sums while using fewer component-gradient evaluations than repeated full-gradient descent. Their costs differ: SVRG needs snapshot passes, SAGA needs memory proportional to the number of components unless structure reduces it, and recursive methods require careful restart logic. They are attractive when the data set is finite and revisited many times, gradients are not dominated by hardware throughput considerations, and high training accuracy matters. They are less compelling in a streaming setting where examples do not recur.

5.3 Momentum and Accelerated Methods

Heavy-ball momentum keeps a velocity,

$$\mathbf{v}_{k+1} = \beta \mathbf{v}_k + \mathbf{g}_k, \qquad \theta_{k+1} = \theta_k - \alpha \mathbf{v}_{k+1},$$

where $0 \leq \beta < 1$. Momentum equations are convention-sensitive. Some implementations store an exponentially weighted average,

$$\mathbf{m}_{k+1} = \beta \mathbf{m}_k + (1-\beta)\mathbf{g}_k, \qquad \theta_{k+1} = \theta_k - \eta \mathbf{m}_{k+1},$$

while others omit $1-\beta$ and compensate through the learning rate. The same printed values of η and β therefore need not define the same update. State the recurrence, not only the optimiser name.

On a one-dimensional quadratic $F(\theta) = \rho\theta^2/2$, heavy-ball momentum gives a second-order recurrence. Its characteristic roots determine whether error contracts monotonically, oscillates while contracting, or grows. The stable region

depends jointly on $\eta\rho$ and β; momentum does not permit an arbitrary gradient-descent step. In several dimensions, one pair of hyperparameters must be stable across the whole curvature spectrum.

Nesterov acceleration should also be identified by its actual sequence. The classical convex proof uses a carefully coupled extrapolation and gradient step. A framework option called "Nesterov momentum" may implement a related stochastic look-ahead update rather than that exact accelerated scheme. It can work well, but the $O(1/k^2)$ theorem belongs to the analysed recurrence and assumptions.

The accumulated direction damps alternating gradients across a narrow valley while preserving progress along a consistent direction. On the rotated quadratic, this can reduce zig-zagging substantially when α and β match the curvature range.

Nesterov acceleration evaluates or applies the gradient at a look-ahead point, depending on the formulation. For convex L-smooth objectives, a classical accelerated method improves the objective-rate bound from $O(1/k)$ to $O(1/k^2)$. Strongly convex variants attain a condition-number dependence involving $\sqrt{\kappa}$ rather than κ. These guarantees depend on the precise update and parameter sequence; calling any momentum implementation "Nesterov" does not import the theorem.

Momentum adds state and another stability boundary. Too much momentum or too large a step produces sustained oscillation. A checkpoint must preserve the velocity. In stochastic training, momentum also filters noise and can lag after a sudden learning-rate or data-distribution change. Inspect update-to-parameter ratios and not only raw gradients, because the velocity determines the actual step.

5.4 Learning-Rate Schedules

A schedule makes α_k vary with iteration, epoch, validation feedback, or a measured statistic. Diminishing schedules support stochastic-approximation convergence by eventually suppressing noise. Piecewise reductions, cosine-shaped schedules, and warm-up phases are engineering policies rather than universal consequences of one theorem.

Warm-up limits early updates when parameter scales, normalisation statistics, or

adaptive moment estimates are not yet representative. It can prevent instability but can also hide a fundamentally excessive peak rate. A restart deliberately raises the learning rate and invalidates any claim that assumes monotone decay; it should be analysed as the implemented non-stationary procedure.

The schedule's independent variable must be explicit. "Ten epochs of warm-up" changes meaning when data size, batch size, gradient accumulation, or distributed worker count changes. Record optimiser updates as well as epochs and examples processed.

5.5 Adaptive Diagonal Methods

Adaptive methods rescale coordinates using gradient history. AdaGrad accumulates squared gradients,

$$\mathbf{s}_k = \mathbf{s}_{k-1} + \mathbf{g}_k \odot \mathbf{g}_k, \qquad \theta_{k+1} = \theta_k - \alpha \frac{\mathbf{g}_k}{\sqrt{\mathbf{s}_k} + \delta},$$

where multiplication, division, and square roots are componentwise and $\delta > 0$ prevents division by zero. AdaGrad has regret and convergence guarantees in convex online and stochastic settings under boundedness assumptions. Its effective step can decay too aggressively when squared gradients accumulate indefinitely.

RMSProp replaces the cumulative sum by an exponential moving average. Adam combines such a second-moment estimate with an exponential moving average of gradients and bias corrections.

One common Adam convention is

$$\begin{aligned}
\mathbf{m}_k &= \beta_1 \mathbf{m}_{k-1} + (1-\beta_1)\mathbf{g}_k, \\
\mathbf{v}_k &= \beta_2 \mathbf{v}_{k-1} + (1-\beta_2)(\mathbf{g}_k \odot \mathbf{g}_k), \\
\widehat{\mathbf{m}}_k &= \frac{\mathbf{m}_k}{1-\beta_1^k}, \qquad \hat{\mathbf{v}}_k = \frac{\mathbf{v}_k}{1-\beta_2^k}, \\
\theta_{k+1} &= \theta_k - \alpha_k \frac{\widehat{\mathbf{m}}_k}{\sqrt{\hat{\mathbf{v}}_k} + \delta}.
\end{aligned}$$

Bias correction matters near the start because both moment estimates are initialised at zero. The stabiliser δ does more than prevent division by zero: when

$\sqrt{\widehat{v}_{k,j}}$ is comparable to or smaller than δ, it sets the effective scale of coordinate j. Placing δ inside rather than outside the square root defines a different algorithm.

The second moment is not a Hessian estimate. It measures recent squared gradients in the chosen coordinates and discards cross-coordinate curvature. Adam can therefore handle persistent differences in coordinate scale while remaining unable to rotate an elongated valley. Its state also makes abrupt changes in loss scale, batch construction, or gradient clipping persist beyond the update where they occur.

To diagnose an adaptive method, log the distributions of $\widehat{m}_{k,j}$, $\sqrt{\widehat{v}_{k,j}}$, and the realised coordinate steps for selected parameter blocks. A stable scalar loss can coexist with a small subset of coordinates whose denominator is tiny and whose updates dominate. Global gradient norm alone will not show that. Adam's original analysis does not justify every practical configuration, and simple convex counterexamples show that some parameter sequences need not converge. AMSGrad and related variants alter the second-moment memory to recover guarantees under their own assumptions. The correct claim names the algorithm variant and theorem, not the family label.

Adaptive diagonal scaling helps when coordinates have persistently different gradient magnitudes or sparse activation. It cannot represent rotated curvature: a diagonal preconditioner cannot exactly undo the off-diagonal structure of the quadratic example. It can also make the meaning of an ℓ_2 gradient penalty differ from multiplicative weight decay. Decoupled weight decay applies shrinkage outside the adaptive gradient transformation; it is not generally equivalent to adding $\frac{\lambda}{2}\|\theta\|^2$ to an adaptively preconditioned objective.

5.6 Coordinate, Subgradient, and Proximal Methods

Coordinate descent updates one coordinate or block at a time. It is effective when a coordinate update is cheap, features are sparse, or a closed-form subproblem exists. For smooth convex objectives, cyclic or randomised variants have convergence guarantees under coordinate-wise smoothness assumptions. Strong coupling between coordinates can make naive cyclic updates slow, and parallel updates can interfere when blocks are not separable.

A convex non-smooth function has a subgradient $\mathbf{g}$ at θ if

$$F(\mathbf{u}) \geq F(\theta) + \mathbf{g}^\top(\mathbf{u} - \theta)$$

for every $\mathbf{u}$. Subgradient descent can optimise objectives such as an absolute-value loss, but its rates are slower and objective values need not decrease each iteration. At a kink, a zero subgradient can certify convex optimality even though an ordinary derivative does not exist.

For a composite objective $F = f + R$ with differentiable f and proximable convex R, proximal gradient uses

$$\theta_{k+1} = \operatorname{prox}_{\alpha_k R}\left(\theta_k - \alpha_k \nabla f(\theta_k)\right),$$

where

$$\operatorname{prox}_{\alpha R}(\mathbf{u}) = \arg\min_{\mathbf{v}} \left\{ R(\mathbf{v}) + \frac{1}{2\alpha}\|\mathbf{v} - \mathbf{u}\|^2 \right\}.$$

For $R(\theta) = \lambda\|\theta\|_1$, the proximal operator is componentwise soft thresholding. Proximal gradient therefore produces exact zeros, unlike an ordinary small gradient step on a smoothed approximation. Under convexity and smoothness of f, it has an $O(1/k)$ objective bound; accelerated variants can reach $O(1/k^2)$. A difficult or inaccurately solved proximal subproblem weakens those guarantees.

For composite optimisation, the ordinary gradient norm is not the right stationarity measure. Define the proximal-gradient mapping

$$\mathbf{G}_\alpha(\theta) = \frac{1}{\alpha}\left[\theta - \operatorname{prox}_{\alpha R}(\theta - \alpha\nabla f(\theta))\right].$$

The update is $\theta_{k+1} = \theta_k - \alpha\mathbf{G}_\alpha(\theta_k)$. A zero mapping is equivalent to the composite first-order condition

$$\mathbf{0} \in \nabla f(\theta) + \partial R(\theta).$$

Its norm therefore provides a stopping certificate that accounts for both the smooth loss and the non-smooth regulariser.

Backtracking can choose α without a known global smoothness constant. Test the smooth part against its quadratic upper model while applying the proximal map at each trial step. Count rejected trials: a method that reports few outer iterations can still perform many gradient or objective evaluations. Accelerated proximal schemes can improve worst-case objective rates, but their extrapolation may make objective values non-monotone. A monotone restart can be an engineering safeguard, though it changes the analysed sequence.

5.7 First-Order Selection Summary

Use full gradient descent when a complete gradient is affordable and accurate. Use stochastic or mini-batch methods when the data sum is too expensive per update. Add momentum when gradients alternate across elongated valleys, but treat its state and stability explicitly. Use adaptive diagonal methods when sparse or uneven coordinate scales justify them, not because they remove all tuning. Use coordinate or proximal methods when separability or non-smooth structure makes their specialised update cheap.

Across the family, the recurring failure modes are incorrect gradients, unsuitable scaling, unstable steps, hidden batch-dependent objective changes, and stopping rules based on noisy quantities. No first-order method repairs a malformed objective.

5.8 Choosing Batch Size, Step Size, and Schedule Together

Batch size and learning rate are coupled through gradient noise and work per update. Increasing the batch usually reduces variance, but it also reduces the number of parameter updates per pass through a fixed data set. Holding epochs fixed therefore changes both the estimator and the number of optimisation decisions. Holding updates fixed changes the number of examples processed. State which budget is fixed before interpreting a batch-size comparison.

At a fixed checkpoint, draw several independent batches and compute gradients $\mathbf{g}^{(1)}, \dots, \mathbf{g}^{(r)}$. Let $\bar{\mathbf{g}}$ be their mean. The diagnostic ratio

$$\frac{\|\bar{\mathbf{g}}\|^2}{r^{-1}\sum_{j=1}^{r}\|\mathbf{g}^{(j)}-\bar{\mathbf{g}}\|^2}$$

is not a universal certificate, but it reveals whether the estimated direction is stable at that point. A very low ratio suggests that more examples, averaging, or a smaller late-stage step may be more useful than additional momentum.

Linear learning-rate scaling with batch size attempts to preserve the cumulative step over a fixed number of processed examples. Square-root scaling targets a different noise argument. Neither is an identity when momentum, adaptive denominators, normalisation, clipping, or data correlation is present. Treat a scaling rule as an initial hypothesis, then check stability and progress under the actual budget.

Schedules also interact with checkpoint semantics. If a run resumes with its update counter reset, a supposedly continuous cosine or warm-up schedule becomes a different procedure. If gradient accumulation changes, an "update" represents a different number of examples. Record updates and examples processed, and restore the schedule counter with optimiser state.

5.9 Online Regret and Offline Optimisation

Adaptive methods are often motivated by online-learning regret bounds. In an online problem, losses f_k arrive sequentially and regret compares cumulative loss with the best fixed decision in hindsight:

$$\text{Regret}_K = \sum_{k=1}^{K} f_k(\theta_k) - \min_{\theta \in \mathcal{C}} \sum_{k=1}^{K} f_k(\theta).$$

Sublinear regret means average regret tends to zero. This is not automatically a convergence result for the last iterate on one fixed empirical objective. Online-to-batch conversions, averaging, and additional assumptions connect the settings, but the conclusion must be translated rather than copied.

AdaGrad's accumulated coordinate scaling is compelling when informative gradients are sparse or coordinates appear at very different frequencies. In a stationary offline problem, its continually growing denominator can eventually make useful steps too small. RMSProp and Adam replace the cumulative sum with exponential memory, improving responsiveness but changing the proof obligations. The practical question is which history a method stores, how that history is initialised and restored, and which result matches that exact

recurrence.

6 Curvature-Aware and Structured Methods

Curvature-aware methods spend more computation per iteration to choose a better-scaled direction. They are attractive when high-accuracy gradients are available, parameters are moderate in number, or reducing expensive objective evaluations matters more than minimising arithmetic per step.

6.1 Newton's Method and Safeguards

Newton's step solves

$$\nabla^2 F(\theta_k)\mathbf{p}_k = -\nabla F(\theta_k), \qquad \theta_{k+1} = \theta_k + \mathbf{p}_k.$$

The equation defines a linear solve, not an instruction to form $[\nabla^2 F]^{-1}$. Dense factorisation costs roughly cubic time and quadratic memory in the parameter dimension, but it is usually more accurate than explicitly computing an inverse. Sparse or structured Hessians can be factorised more cheaply, and matrix-free methods can approximately solve the system using only Hessian-vector products.

For a positive-definite Hessian, conjugate gradient can minimise the local quadratic model over an expanding Krylov subspace. Truncated Newton methods stop the inner solve when its residual is small enough for the outer iteration, or when negative curvature is detected. Solving every early Newton system to machine precision wastes work because the local model itself is inaccurate far from a solution. An *inexact Newton forcing sequence* ties inner accuracy to outer progress, allowing loose early solves and increasingly accurate later ones.

The linear-system residual

$$\mathbf{r}_k = \nabla^2 F(\theta_k)\mathbf{p}_k + \nabla F(\theta_k)$$

should be logged alongside the outer gradient. A small outer step with a large residual may indicate an inner-solver failure rather than convergence. Preconditioning the Newton system can be decisive because the same curvature spread

that slows gradient descent also slows an unpreconditioned Krylov solve.

For the rotated quadratic, the Taylor model is exact and $\mathbf{p}_k = \theta^* - \theta_k$, so one exact Newton step reaches the minimiser from any start. This dramatic result depends on a constant positive-definite Hessian and an exact linear solve. It is an explanatory endpoint, not a typical cost profile.

Near a strict minimiser where the Hessian is Lipschitz continuous and nonsingular, Newton's method can converge quadratically: the number of correct digits can roughly double per iteration. Far from that region, an indefinite Hessian can produce an ascent direction, a nearly singular Hessian can produce an enormous step, and the quadratic model can be inaccurate.

Damped Newton combines the Newton direction with a line search. Modified Newton adds diagonal damping or changes an indefinite factorisation to obtain a descent direction. These safeguards provide global progress under additional assumptions, followed by fast local convergence. They also change the algorithm, so a report should say whether the step was pure, damped, truncated, or regularised.

For a descent direction $\mathbf{p}_k$ with $\nabla F_k^\top \mathbf{p}_k < 0$, the Armijo condition accepts a step α_k when

$$F(\theta_k + \alpha_k \mathbf{p}_k) \leq F(\theta_k) + c_1 \alpha_k \nabla F_k^\top \mathbf{p}_k,$$

where $0 < c_1 < 1$. It requires decrease proportional to the first-order prediction, not merely any decrease. The strong Wolfe curvature condition additionally requires

$$\left|\nabla F(\theta_k + \alpha_k \mathbf{p}_k)^\top \mathbf{p}_k\right| \leq c_2 \left|\nabla F_k^\top \mathbf{p}_k\right|,$$

with $c_1 < c_2 < 1$. This prevents an accepted step from being so short that it learns almost nothing about curvature along the line.

The curvature condition is especially important for BFGS because, under suitable conditions, it gives $\mathbf{y}_k^\top \mathbf{s}_k > 0$ and preserves positive definiteness of the approximation. In noisy optimisation, repeated evaluations at nearby trial points may differ because of sampling rather than step quality. Use a fixed batch during a line search or choose a stochastic line-search analysis that explicitly models

the noise.

For regularised logistic regression, the Hessian $\mathbf{X}^\top \mathbf{W}\mathbf{X}/n + \lambda \mathbf{I}$ is positive definite when all directions receive positive ℓ_2 regularisation. Newton steps are then well defined, but forming and factorising a dense d-by-d Hessian can be prohibitive. As fitted probabilities approach zero or one, entries of $\mathbf{W}$ shrink and data curvature can become weak; regularisation supplies a lower curvature floor.

6.2 Gauss–Newton and Natural-Gradient Ideas

For non-linear least squares, Gauss–Newton solves

$$\mathbf{J}_r^\top \mathbf{J}_r \, \mathbf{p} = -\mathbf{J}_r^\top r.$$

The matrix is positive semidefinite, avoiding the negative curvature contributed by the residual-Hessian term omitted from the exact Hessian. This is an advantage when the omitted term is small and a limitation when that term contains important local geometry. Levenberg–Marquardt adds damping,

$$\left(\mathbf{J}_r^\top \mathbf{J}_r + \gamma \mathbf{I}\right) \mathbf{p} = -\mathbf{J}_r^\top r,$$

interpolating between a Gauss–Newton step and a scaled gradient step as γ changes. The damping parameter can be controlled by trust-region-style model agreement.

Natural gradient replaces Euclidean distance in parameter space by a local measure of change in the model's predictive distribution. With a Fisher information matrix $\mathbf{G}(\theta)$, the idealised direction solves

$$\mathbf{G}(\theta_k)\mathbf{p}_k = -\nabla F(\theta_k).$$

This can reduce sensitivity to smooth reparameterisation, but exact Fisher matrices are generally too large. Diagonal, block, Kronecker-factored, or empirical approximations define different algorithms and invariance properties. Damping is again required when the metric is singular or poorly estimated. "Second order" should therefore identify the matrix actually used, not imply an exact Hessian.

6.3 Trust-Region Methods

At iterate θ_k, define a quadratic model

$$m_k(\mathbf{p}) = F(\theta_k) + \nabla F_k^\top \mathbf{p} + \frac{1}{2}\mathbf{p}^\top \mathbf{B}_k \mathbf{p}$$

and approximately minimise it subject to $\|\mathbf{p}\| \le \Delta_k$. The trust-region radius Δ_k limits how far the local model is allowed to speak. The step can remain well defined even when $\mathbf{B}_k$ is indefinite because the bounded subproblem cannot run to negative infinity.

After evaluating the trial point, compare actual and predicted reduction:

$$\rho_k = \frac{F(\theta_k) - F(\theta_k + \mathbf{p}_k)}{m_k(\mathbf{0}) - m_k(\mathbf{p}_k)}.$$

When ρ_k is close to one, the model predicted the objective well and the radius may grow. When it is small or negative, reject or shorten the step and shrink the radius. The exact thresholds are algorithm parameters, but the principle is structural: model agreement, not objective decrease alone, controls future step freedom.

The Cauchy point, obtained by minimising the model along the negative gradient within the radius, supplies a baseline amount of predicted decrease. Practical subproblem solvers need not find the exact trust-region minimiser, but convergence analyses normally require at least a declared fraction of this baseline. Log ρ_k, Δ_k, predicted reduction, actual reduction, and subproblem termination. A radius collapsing towards zero while the gradient remains large points to a poor model, inconsistent objective evaluations, or bad scaling.

6.4 Quasi-Newton Methods

Let

$$\mathbf{s}_k = \theta_{k+1} - \theta_k, \qquad \mathbf{y}_k = \nabla F(\theta_{k+1}) - \nabla F(\theta_k).$$

These vectors satisfy the secant relation $\nabla^2 F\, \mathbf{s}_k \approx \mathbf{y}_k$. If $\mathbf{H}_k$ approximates the inverse Hessian and $\rho_k = (\mathbf{y}_k^\top \mathbf{s}_k)^{-1}$, the inverse-BFGS update is

$$\mathbf{H}_{k+1} = \left(\mathbf{I} - \rho_k \mathbf{s}_k \mathbf{y}_k^\top\right) \mathbf{H}_k \left(\mathbf{I} - \rho_k \mathbf{y}_k \mathbf{s}_k^\top\right) + \rho_k \mathbf{s}_k \mathbf{s}_k^\top.$$

If $\mathbf{H}_k$ is positive definite and $\mathbf{y}_k^\top \mathbf{s}_k > 0$, the update remains positive definite and $-\mathbf{H}_{k+1}\nabla F$ is a descent direction. A suitable Wolfe line search gives this curvature condition for smooth deterministic problems. Noisy gradient differences can violate it even when the underlying objective is well behaved. Implementations may skip, damp, or regularise a curvature pair; count those events because frequent rejection means the approximation is not learning useful curvature.

Full BFGS stores a dense matrix and therefore needs $O(d^2)$ memory. L-BFGS stores only the latest m pairs and applies the implicit inverse approximation with a two-loop recursion, using $O(md)$ memory. It is a strong baseline for smooth deterministic or large-batch problems when repeated objective and gradient evaluations are affordable. It is less natural when every gradient is based on an unrelated noisy batch, because $\mathbf{y}_k$ then mixes curvature change with sampling change.

Superlinear local convergence belongs to BFGS under smoothness, positive-definite local curvature, a suitable line search, and other technical conditions. L-BFGS with fixed memory has different theory and should not inherit the full-memory claim automatically.

6.5 Hessian-Vector Products and Preconditioning

Many large problems can compute $\nabla^2 F(\theta)\mathbf{v}$ without materialising the Hessian. Automatic differentiation can obtain this product at a cost comparable to a small number of gradient evaluations. Krylov methods then approximately solve the Newton system in a subspace, yielding truncated Newton or Hessian-free methods.

The inner solve introduces another tolerance. Solving the Newton system too accurately early wastes work because the local model is imperfect; solving it too loosely near a solution can destroy fast convergence. An adaptive forcing rule ties inner accuracy to the outer gradient norm.

Preconditioning transforms the linear system so that its effective eigenvalues cluster. The rotated quadratic shows the ideal case: an exact inverse-curvature

preconditioner reduces the condition number to one. In practice, diagonal, block, low-rank, or problem-structured approximations trade setup cost against iteration count. Evaluate the total cost, including constructing and updating the preconditioner.

Compare curvature-aware methods using declared operations rather than iteration count alone:

Method	Stored State	Main Per-Step Work	Globalisation	Diagnostic to Retain
Damped Newton	Hessian or factorisation	Hessian construction plus linear solve	Line search	Linear residual and accepted step
Truncated Newton	Vectors for a Krylov solve	Gradient and Hessian-vector products	Line search or trust region	Inner residual and negative-curvature event
Gauss–Newton	Jacobian products or normal matrix	Least-squares linear solve	Damping or trust region	Residual size and model agreement
BFGS	Dense inverse approximation	Gradient plus $O(d^2)$ update	Usually Wolfe line search	$\mathbf{y}^\top \mathbf{s}$ and rejected pairs
L-BFGS	m curvature pairs	Gradient plus $O(md)$ recursion	Usually line search	Pair acceptance and history size
Trust-region Newton	Model plus subproblem state	Approximate constrained quadratic solve	Radius update is intrinsic	Radius, ρ_k, and predicted reduction

The table does not select a winner. It makes hidden costs visible. If an objective evaluation requires an expensive simulation, a method that uses more linear

algebra but fewer rejected evaluations may dominate. If gradients arrive from a high-throughput stochastic pipeline, a dense factorisation may be irrelevant regardless of its iteration rate.

6.6 Curvature-Aware Selection Summary

Use Newton or trust-region methods for moderate, smooth problems when derivatives and linear solves are reliable. Use Gauss–Newton when a residual structure makes its approximation accurate. Use BFGS or L-BFGS when gradients are reliable but Hessians are too expensive. Use Hessian-vector products and iterative solves when curvature is useful but dense storage is impossible.

The family can fail through indefinite or singular curvature, noisy gradient differences, inaccurate linear solves, or a local model trusted too far. Safeguards are part of the method, not optional decorations.

6.7 A Worked Curvature-Aware Decision

Suppose regularised logistic regression has $d = 2,000$ dense parameters and $n = 100,000$ examples. A dense Hessian contains four million entries, which may be manageable, but forming it requires aggregating curvature over the data and factorising it costs far more than one gradient. Full Newton is plausible if very high accuracy is required and only a few fits are needed. It is less attractive inside a large hyperparameter search.

L-BFGS uses full gradients but stores only a short curvature history. It is a strong first baseline when deterministic passes are affordable. If each full pass is expensive, a truncated Newton method can use Hessian-vector products and stop its inner solve early. The choice depends on measured operations: gradient passes, Hessian-vector products, line-search evaluations, memory, and achieved residual.

Near the solution, logistic probabilities close to zero or one contribute little data curvature because $p_i(1-p_i)$ is small. Regularisation keeps the Hessian bounded below when every direction is penalised. If an intercept is unpenalised and the data provide weak curvature in that direction, the Newton system can remain poorly conditioned. A block preconditioner or separate treatment of the intercept may be more effective than increasing global damping.

Run a staged comparison. First, verify that methods agree on final objective

and gradient norm on a reduced deterministic data set. Second, record time and oracle calls to several residual thresholds. Third, perturb the starting point and regularisation strength. A method that wins only from one favourable start or at one loose threshold has not established a general advantage.

6.8 Model Quality and Step Quality

Curvature methods fail in two separable ways. The model may be poor, or the model may be solved poorly. Trust-region agreement ρ_k tests the first. The subproblem or Newton-system residual tests the second. A low ρ_k with an accurate subproblem solve says the local quadratic was not predictive at that radius. A high inner residual says little about model quality because the proposed step did not minimise the model adequately.

Negative curvature needs an explicit policy. Modified factorisations alter an indefinite Hessian to obtain a descent direction. Trust-region and truncated-CG methods can exploit a detected negative-curvature direction while bounding the step. Gauss–Newton discards a class of negative-curvature terms by construction. These choices solve different local models and should produce different diagnostic records.

7 Constraints, Duality, and Regularisation

Constraints encode feasible decisions; regularisers change preference among feasible parameters; penalties trade violation against objective value. Dual variables expose the marginal value of constraints and support optimality certificates when the problem structure permits them.

7.1 Feasible Sets and Projections

Write a general constrained problem as

$$\begin{aligned} \min_{\theta} \quad & F(\theta) \\ \text{subject to} \quad & g_j(\theta) \le 0, \quad j = 1, \dots, m, \\ & h_r(\theta) = 0, \quad r = 1, \dots, p. \end{aligned}$$

An equality removes motion normal to its local surface. An inactive inequality

permits local motion in every sufficiently small direction; an active inequality restricts directions that would cross its boundary. Bounds and simplex constraints are special cases whose projections can be cheap. A general non-linear feasible set may be non-convex, disconnected, or expensive even to test.

The first-order object is no longer the unconstrained gradient. At a feasible point, a descent direction must also lie in the feasible tangent cone. If the negative gradient points outside the set, the constrained optimum can have a non-zero gradient balanced by the boundary. This is why stopping on $\|\nabla F\|$ alone is wrong for constrained problems.

For a closed convex set, the Euclidean projection

$$\Pi_{\mathcal{C}}(\mathbf{u}) = \arg\min_{\mathbf{v}\in\mathcal{C}} \frac{1}{2}\|\mathbf{v}-\mathbf{u}\|^2$$

is unique. Projection onto a box is componentwise clipping; projection onto an affine set is a linear solve; projection onto a probability simplex requires a sorting or selection procedure. Projected gradient uses $\theta_{k+1} = \Pi_{\mathcal{C}}(\theta_k - \alpha\nabla F_k)$. Its stationarity measure is the projected-gradient mapping

$$\mathbf{G}_{\alpha}^{\mathcal{C}}(\theta) = \frac{1}{\alpha}\left[\theta - \Pi_{\mathcal{C}}(\theta - \alpha\nabla F(\theta))\right].$$

This is a proximal-gradient mapping for the indicator function of $\mathcal{C}$. It connects projected and proximal methods and gives a certificate that respects the boundary. Under convexity and smoothness assumptions, projected-gradient convergence parallels unconstrained gradient descent. Clipping parameters after an arbitrary optimiser update is only projected optimisation if the clipping is the correct projection and the complete update is analysed with it.

7.2 Lagrangians, KKT Conditions, and Duality

Using the sign convention $g_j(\theta) \le 0$, define the Lagrangian

$$\mathcal{L}(\theta,\gamma,\nu) = F(\theta) + \sum_{r=1}^{p}\gamma_r h_r(\theta) + \sum_{j=1}^{m}\nu_j g_j(\theta), \qquad \nu \ge \mathbf{0}.$$

The dual function is the best lower bound obtained for fixed multipliers:

$$d(\gamma, \nu) = \inf_{\theta} \mathcal{L}(\theta, \gamma, \nu).$$

For any feasible θ, the equality terms vanish and each inequality term is non-positive, so

$$d(\gamma, \nu) \leq \mathcal{L}(\theta, \gamma, \nu) \leq F(\theta).$$

Maximising d over dual-feasible multipliers therefore gives the dual problem and a lower bound on the primal optimum. This is *weak duality* and requires no convexity. For a minimisation problem, a feasible primal value and a dual-feasible value bracket the optimum. Their difference is a duality gap and can be a rigorous stopping certificate when both values are computed for the same primal–dual pair.

Strong duality means the best dual bound equals the primal optimum. It does not hold for every problem. In a convex problem, Slater's condition — existence of a point satisfying the equalities and strictly satisfying the convex inequalities, under the standard affine-equality setup — is a common sufficient condition. Other constraint qualifications apply in other settings. State the qualification rather than treating zero duality gap as automatic.

With the convention above, the KKT conditions at a candidate $(\theta^*, \gamma^*, \nu^*)$ are

$$\begin{aligned}
\nabla_\theta \mathcal{L}(\theta^*, \gamma^*, \nu^*) &= \mathbf{0}, && \text{stationarity,} \\
g_j(\theta^*) &\leq 0, && \text{inequality feasibility,} \\
h_r(\theta^*) &= 0, && \text{equality feasibility,} \\
\nu_j^* &\geq 0, && \text{dual feasibility,} \\
\nu_j^* g_j(\theta^*) &= 0, && \text{complementary slackness.}
\end{aligned}$$

Complementary slackness says that an inactive inequality has zero multiplier, while an active inequality may carry a non-zero shadow price. It does not say that every active constraint has a positive multiplier; degeneracy can produce active constraints with zero multipliers.

The logical status matters. Under an appropriate constraint qualification, KKT conditions are necessary for a local optimum. For a convex differentiable ob-

jective with convex inequality functions and affine equality constraints, they are sufficient as well; with strong duality they characterise a primal–dual optimum. In a non-convex problem, a KKT point can be a local maximum or saddle on the feasible set.

Constraint qualifications rule out pathological descriptions of the same feasible geometry. For example, the constraint $\theta^2 \leq 0$ defines the single feasible point $\theta = 0$, but its gradient vanishes there, so ordinary multiplier arguments lose directional information. Failure to find KKT multipliers may indicate a failed constraint qualification rather than absence of an optimum.

Consider projecting a scalar target a onto the non-negative half-line:

$$\min_{\theta} \ \frac{1}{2}(\theta - a)^2 \quad \text{subject to} \quad -\theta \leq 0.$$

The Lagrangian is

$$\mathcal{L}(\theta, \nu) = \frac{1}{2}(\theta - a)^2 - \nu\theta, \qquad \nu \geq 0.$$

Stationarity gives $\theta - a - \nu = 0$. If $a \geq 0$, the unconstrained solution is feasible, so $\theta^* = a$ and $\nu^* = 0$. If $a < 0$, the boundary is active: $\theta^* = 0$ and $\nu^* = -a > 0$. The multiplier measures how much the optimum would improve if the boundary were relaxed locally. The solution is exactly $\theta^* = \max(a, 0)$, the projection and also the proximal operator of the indicator of the half-line.

This one-dimensional example contains the full KKT logic: an interior point has no active force, while a boundary optimum balances the objective gradient against a feasible-set normal. Higher-dimensional projected, proximal, and primal–dual methods build on the same geometry.

7.3 Penalty, Barrier, and Augmented-Lagrangian Methods

For equality constraints $\mathbf{h}(\theta) = \mathbf{0}$, a quadratic penalty solves a sequence of unconstrained problems

$$F_\rho(\theta) = F(\theta) + \frac{\rho}{2}\|\mathbf{h}(\theta)\|^2.$$

Large ρ discourages violation, but finite ρ does not usually enforce exact feasibility. Driving ρ towards infinity can make the Hessian badly conditioned: curvature normal to the constraint grows with ρ, while tangential curvature need not. A solver may appear to stall because the penalty formulation created an increasingly narrow valley.

An interior logarithmic barrier for inequalities $g_j(\theta) < 0$ is

$$F_\tau(\theta) = F(\theta) - \tau \sum_{j=1}^{m} \log\bigl(-g_j(\theta)\bigr), \qquad \tau > 0.$$

The barrier is defined only at strictly feasible points and diverges at the boundary. Solving a sequence with $\tau \downarrow 0$ follows a central path towards the constrained solution under suitable conditions. The barrier does not "soften" a constraint in the same way as a finite penalty: it prevents crossing the boundary but requires an interior start or a separate feasibility phase.

The augmented Lagrangian combines multipliers with a quadratic penalty:

$$\mathcal{L}_\rho(\theta, \gamma) = F(\theta) + \gamma^\top \mathbf{h}(\theta) + \frac{\rho}{2} \|\mathbf{h}(\theta)\|^2.$$

After an approximate inner minimisation, update

$$\gamma_{k+1} = \gamma_k + \rho_k \mathbf{h}(\theta_{k+1}).$$

The multiplier accumulates information about the force needed to enforce feasibility, so the method can achieve accurate constraints without sending ρ to an extreme value. Its diagnostics must include both inner optimality and outer feasibility; a small inner gradient of $\mathcal{L}_\rho$ does not prove that the original constraints are satisfied.

7.4 Regularisation as Geometry and Statistical Choice

An ℓ_2 penalty shrinks parameters continuously and adds isotropic curvature when applied to every direction. In logistic regression, it changes the Hessian from $\mathbf{X}^\top \mathbf{W} \mathbf{X}/n$ to that matrix plus $\lambda \mathbf{I}$. The minimum eigenvalue increases by λ, while the maximum also increases by λ, so the condition number can improve

substantially when data curvature has nearly flat directions.

An ℓ_1 penalty promotes exact zeros through non-smooth geometry and is naturally handled by coordinate or proximal methods. It does not make correlated predictors uniquely identifiable in every case; several sparse solutions can fit nearly equally well. Group penalties, nuclear norms, and structured regularisers encode other forms of simplicity, each with its own proximal map or solver structure.

Regularisation has three distinguishable effects: it changes the statistical preference among models, it can improve numerical conditioning, and it can encode a prior or constraint-like belief. These effects can align, but they are not the same claim. Select λ using a declared validation design rather than choosing the value that makes the training optimiser easiest.

For the recurring logistic example, positive λ prevents coefficients from diverging on linearly separable data when all relevant directions are penalised. Without regularisation, the logistic loss can approach zero as coefficient norms grow while no finite minimiser exists. A solver then reports ever-larger parameters and smaller gradients without a numerical bug. The failure is a property of the unregularised formulation.

Constrained solvers should report a scaled KKT residual, not only objective value. Separate at least:

$$
\begin{aligned}
r_{\text{stat}} &= \left\| \nabla_\theta \mathcal{L} \right\|, \\
r_{\text{eq}} &= \|\mathbf{h}(\theta)\|, \\
r_{\text{ineq}} &= \|[\mathbf{g}(\theta)]_+\|, \\
r_{\text{comp}} &= \|\nu \odot \mathbf{g}(\theta)\|, \\
r_{\text{dual}} &= \|[-\nu]_+\|.
\end{aligned}
$$

Here $[\cdot]_+$ takes the positive part componentwise. Tolerances need units and scales. A violation of 10^{-4} metres and a violation of 10^{-4} kilograms are not comparable merely because their numbers match. Scale constraints by meaningful reference magnitudes or report both absolute and relative residuals.

Before blaming the optimiser, test feasibility independently. Minimise a violation measure or solve a Phase I problem. If the best attainable violation remains large, the original constraints may conflict or the model may be unable to satisfy

them. Increasing a penalty cannot make an infeasible problem feasible.

Regularisation diagnostics are different. A penalty coefficient controls a modelling trade-off and is often selected by held-out evidence; a constraint tolerance describes acceptable feasibility. Do not tune a hard safety requirement as if it were an ordinary predictive hyperparameter unless the application explicitly permits that trade-off.

7.5 Constraints and Regularisation Failure Modes

The common failures are infeasible requirements, silently scaled penalties, incorrect projections, and invalid dual claims. Diagnose feasibility before tuning an optimiser. Report constraint residuals in their physical units. Compare penalty terms with the data loss under the exact averaging convention. If a solution sits on a boundary, use constrained stationarity rather than expecting a zero raw gradient.

7.6 Worked KKT Example: Projection Onto a Simplex

Let $\mathbf{a} \in \mathbb{R}^d$ and project it onto the probability simplex:

$$\min_{\theta} \frac{1}{2}\|\theta - \mathbf{a}\|^2 \quad \text{subject to} \quad \mathbf{1}^\top \theta = 1, \qquad \theta \geq \mathbf{0}.$$

Write the inequalities as $-\theta_j \leq 0$. With equality multiplier γ and inequality multipliers $\nu_j \geq 0$, stationarity is

$$\theta - \mathbf{a} + \gamma \mathbf{1} - \nu = \mathbf{0}.$$

Complementary slackness gives $\nu_j \theta_j = 0$. If $\theta_j > 0$, then $\nu_j = 0$ and $\theta_j = a_j - \gamma$. If $\theta_j = 0$, dual feasibility requires $\gamma \geq a_j$. Therefore

$$\theta_j^* = \max(a_j - \gamma, 0),$$

where γ is chosen so the components sum to one. Sorting the entries of $\mathbf{a}$ reveals the active set and permits an efficient exact projection. Componentwise clipping alone is wrong: it enforces non-negativity but generally destroys the sum constraint.

This example supplies a diagnostic. After projection, check non-negativity, the sum residual, stationarity on positive components, and the threshold inequality on zero components. A small Euclidean distance to the input is not enough if the result is infeasible.

7.7 Choosing a Constrained Method

Active-set methods estimate which inequalities bind, solve a reduced equality-constrained problem, and update the active set. They are effective when only a modest number of constraints are active and the set stabilises. They can struggle with degeneracy or frequent changes in activity.

Sequential quadratic programming builds a quadratic model of the Lagrangian with linearised constraints. Under accurate derivatives, a suitable constraint qualification, and second-order conditions, it can converge rapidly near a solution. Globalisation requires a merit function, filter, or trust region that balances objective reduction against feasibility.

Interior-point methods introduce slack variables and barrier terms, then solve coupled primal–dual systems while reducing a barrier parameter. Their iterates need not monotonically improve the original objective. Progress is measured through primal residuals, dual residuals, complementarity, and movement along the central path.

Augmented-Lagrangian and splitting methods are attractive when constraints or objective terms decompose. Their inner subproblems and multiplier updates create several tolerances. Tightening every inner solve from the start wastes work; leaving errors large near termination prevents the outer KKT residual from shrinking. Record the tolerance schedule as part of the algorithm.

Method selection begins with structure: Is projection closed-form? Are constraints convex? Is strict feasibility available? Are Jacobians sparse? Does the problem decompose? Can a linearised constraint model be solved reliably? The number of constraints alone does not answer these questions.

8 Non-Convex Optimisation in Machine Learning

Neural-network objectives are non-convex because layers compose nonlinear maps and because parameter symmetries create many equivalent representa-

tions. The practical goal is usually a parameter state with acceptable validation and operational behaviour, not a certificate of global optimality.

8.1 What the Guarantees Become

For a smooth objective bounded below, gradient descent with a suitable step can guarantee that some iterate has a small gradient norm. Stochastic variants can obtain expected stationarity bounds under assumptions on estimator bias, variance, smoothness, and schedules. These are first-order results. They do not say the stationary point is a local or global minimum.

A second-order stationary point also has no materially negative Hessian eigenvalue. Under additional regularity and randomness assumptions, perturbed methods can escape strict saddles and approach approximate second-order stationarity. Neural objectives can contain degenerate saddles and broad nearly flat regions that do not fit the strict-saddle model cleanly.

Training loss can decline while validation loss rises, and validation metrics can improve while the gradient remains large. Optimisation convergence and generalisation are related through the learning procedure but are not interchangeable. Report both.

For a twice differentiable unconstrained objective, a candidate local minimum satisfies the first- and second-order necessary conditions

$$\nabla F(\theta^*) = \mathbf{0}, \qquad \nabla^2 F(\theta^*) \succeq \mathbf{0}.$$

A practical approximate second-order certificate is

$$\|\nabla F(\theta)\| \leq \varepsilon_g, \qquad \lambda_{\min}(\nabla^2 F(\theta)) \geq -\varepsilon_H.$$

The first condition rules out a large first-order descent direction. The second rules out a sufficiently negative-curvature direction. Together they identify an approximate second-order stationary point, not a global minimum. Estimating the smallest eigenvalue can be done approximately with Hessian-vector products and a Krylov method, but the estimate has its own tolerance and stochastic error.

If the Hessian has a negative eigenvalue at a nearly stationary point, moving a

small distance in the associated eigenvector direction decreases the quadratic model in one of the two signs. Trust-region and negative-curvature methods can exploit this. A pure gradient method may move slowly because its first-order signal is already small.

8.2 Saddles, Plateaux, Symmetries, and Cliffs

At a saddle, the gradient can be small while the Hessian has a negative eigenvalue. In high dimensions, there are many possible curvature directions, so a scalar gradient norm alone hides structure. Negative-curvature methods, noise, or momentum may help escape, but a noisy loss trace is not proof that a particular saddle caused delay.

Parameter symmetries produce equivalent models. Permuting hidden units or rescaling adjacent homogeneous layers can preserve predictions while changing parameter coordinates. Consequently, distance between parameter vectors can be a poor measure of model difference, and the Hessian can have flat directions unrelated to poor predictive behaviour.

Exploding gradients usually indicate a steep or numerically unstable region, an excessive step, or unstable forward activations. Vanishing gradients can arise from saturation, depth, poor scaling, or a genuinely flat region. Gradient clipping bounds an update statistic and can stabilise training, but it changes the gradient estimator and introduces bias. State the clipping rule and threshold when making convergence claims.

These geometries need different diagnoses. A strict saddle has a negative-curvature direction. A plateau has small gradients and small curvature over a region. A symmetry creates directions in parameter space that change parameters without changing the represented function. Saturation can make derivatives small because an activation or probability is in an extreme regime. A cliff has a rapid gradient increase over a short distance. Calling all of them “flat loss landscape” discards the distinction that determines the next test.

Neural-network symmetries are common. Permuting hidden units and applying the inverse permutation to the next layer leaves the function unchanged. Positively homogeneous activations can permit one layer to be scaled up while the next is scaled down. These transformations create equivalent parameter solutions and can change Hessian eigenvalues or Euclidean notions of flatness

without changing predictions. A claim that one optimiser finds “flatter minima” therefore needs a parameterisation-aware measure and held-out evidence; raw Hessian sharpness alone is not invariant.

Use interventions to distinguish causes. Perturb along an estimated most-negative-curvature direction to test a saddle. Rescale inputs or inspect activation derivatives to test saturation. Compare prediction changes under parameter perturbations to distinguish functional flatness from parameter symmetry. Evaluate a line segment between solutions only after aligning obvious permutations; a high-loss midpoint can otherwise be an artefact of mismatched units.

8.3 Mini-Batch Noise and Training Dynamics

Stochasticity comes from sample selection, augmentation, dropout-like mechanisms, distributed asynchrony, and finite-precision arithmetic. The resulting gradient need not be an unbiased estimate of one fixed objective. For example, stateful normalisation can couple examples within a batch, while augmentation defines an expectation over transformations as well as data.

Noise can help move through flat or saddle regions, but too much noise creates a high error floor. Increasing batch size, reducing the learning rate, or averaging can improve late-stage accuracy. These changes also alter throughput and sometimes the implicit model-selection behaviour. Treat “noise is regularisation” as a mechanism to test, not a substitute for measuring validation performance.

Noise can help an iterate leave some unstable saddles, but the claim needs conditions on the noise directions and local geometry. Mini-batch noise is anisotropic and state-dependent; it does not supply uniform random exploration. In a narrow valley, the same noise can prevent accurate settling. Measure its scale relative to the gradient and curvature instead of treating it as automatically beneficial.

Gradient clipping changes the estimator. Global-norm clipping replaces $\mathbf{g}$ by

$$\tilde{\mathbf{g}} = \mathbf{g} \min\left(1, \frac{\tau}{\|\mathbf{g}\|}\right).$$

It can prevent a rare large update from destroying a run, but generally introduces bias because $\mathbb{E}[\tilde{\mathbf{g}}] \neq \nabla F$. Log the unclipped norm, clipped norm, and

clipping frequency. If clipping occurs on most steps, it is part of the effective optimisation geometry rather than an occasional safety guard.

Other practical transformations — per-example clipping, quantisation, asynchronous staleness, gradient accumulation, and loss scaling — likewise define the update that must be analysed. Automatic differentiation can compute the pre-transformation gradient exactly while the optimiser follows a materially different direction.

8.4 Optimiser Choice and Generalisation

Two optimisers can reach similar training losses with different parameter norms, margins, or trajectories and then show different validation behaviour. That observation does not imply a universal generalisation ranking. Optimiser, schedule, batch construction, regularisation, stopping, and initialisation form one learning procedure.

Adaptive methods often make rapid initial progress, while momentum SGD can remain competitive after careful scheduling. This is an empirical comparison, not a theorem that transfers between architectures and data sets. Compare methods at matched budgets, tune each fairly, repeat stochastic trials, and preserve failed runs.

Decoupled weight decay deserves explicit treatment. For ordinary SGD, multiplicative shrinkage can be made equivalent to an ℓ_2 gradient penalty after accounting for the learning rate. Under coordinate-adaptive scaling, the two operations differ. Record whether the implementation regularises the objective or applies a separate decay update.

Training objective and generalisation can be correlated without one determining the other. Two procedures can finish at nearly equal training loss and different validation performance because they reached different functions, because regularisation or data order differed, or because the validation estimate is noisy. Conversely, a procedure with higher training loss can generalise better because it stopped earlier or imposed a stronger inductive bias.

Do not infer a universal generalisation ranking from one architecture and split. Hold the objective, data order policy, augmentation, update budget, and selection rule fixed; tune each method over a declared space; repeat over seeds; and report the joint distribution of training and held-out outcomes. If optimiser

choice changes the effective regularisation — for example through decoupled weight decay, clipping, or early stopping — say so rather than attributing the result only to “better optimisation”.

Validation-based checkpoint selection is an outer adaptive process. Reporting the maximum validation score across many checkpoints and trials without accounting for that search overstates evidence. Preserve the full trial history and use an untouched test set or nested design for a final comparison.

8.5 Hyperparameter Search as an Outer Loop

The outer objective can be discontinuous, noisy, expensive, and conditional on training budget. Random search is a strong baseline when only a subset of hyperparameters materially affects performance, because a grid wastes trials by tying all coordinates to the same resolution. Model-based search, bandit allocation, and population methods add assumptions and state; compare them against random search at the same resource budget.

Early termination of poor trials changes the target from final performance at a fixed budget to performance under a resource-allocation policy. That can be useful, but the reported comparison must name the policy. Warm-starting trials saves work while coupling their randomness and trajectory. Reusing a validation set across many adaptive trials increases selection bias.

The inner optimiser need not be solved to high precision for every outer trial. A cheap, noisy inner solve may rank configurations well enough, but it can also favour methods that learn quickly early rather than methods that finish well. Multi-fidelity design should define which fidelity dimensions vary: data size, model size, epochs, precision, or another resource.

A seed changes initial parameters, data order, stochastic layers, augmentation, and sometimes non-deterministic kernels. Separate these sources when diagnosing sensitivity. Reusing one integer does not guarantee identical execution across hardware or software environments, and changing all sources together does not reveal which one matters.

For repeated runs, report individual outcomes as well as a summary. With few seeds, a mean and standard deviation can hide skew or a failed-run mode. Show the number of finite runs, median, range or quantiles, and the pre-declared treatment of failures. Do not silently discard divergent runs; stability is part of op-

timiser performance.

Compare trajectories at aligned work budgets, not only their selected endpoints. Time-to-threshold distributions reveal whether a method is consistently useful or occasionally excellent. A paired design using the same data split and, where appropriate, matched data-order seeds can reduce nuisance variation, but preserve any randomness that is part of the method itself.

8.6 A Defensible Non-Convex Claim

A defensible report can state that, across five declared seeds and a fixed update budget, the procedure reached finite training losses, reduced the diagnostic gradient norm, and selected a checkpoint by the pre-registered validation rule. It should not claim convergence to the optimum without a certificate that the non-convex setting rarely supplies.

The principal failure modes are silent divergence, convergence to a stationary but useless solution, validation overfitting, irreproducible sensitivity to seeds, and unfair optimiser comparisons. The diagnostic workflow is designed to distinguish them.

9 Diagnostics and an End-to-End Workflow

Optimisation is observable when you record the objective's components, gradient and update scales, feasibility, numerical health, data order, and validation behaviour. A single loss curve is not enough to identify most failures.

Organise diagnostics into four layers. A failure at an earlier layer invalidates conclusions from later ones:

1. **Implementation evidence:** the objective, derivatives, projection or proximal map, and parameter update match their mathematical definitions.
2. **Optimisation evidence:** the run reduces an appropriate residual or objective under a declared resource budget and remains numerically finite.
3. **Statistical evidence:** held-out performance is estimated under a valid split or resampling design and variation across samples or seeds is reported.
4. **Operational evidence:** the selected checkpoint satisfies latency, memory, robustness, fairness, safety, or other deployment requirements.

A low training loss is optimisation evidence, not proof of the other three layers. A strong test metric does not retroactively prove that the optimiser converged. Keeping the layers separate makes an apparently contradictory result intelligible: a deliberately early-stopped model can have incomplete optimisation evidence and strong statistical evidence.

9.1 Step 1: Freeze the Optimisation Contract

Write the objective mathematically and in executable terms. State whether losses are summed or averaged, which parameters are regularised, how weights are normalised, how constraints are applied, and which randomness enters. Record the data snapshot, split identifiers, preprocessing, parameter initialisation, numeric precision, batch construction, optimiser state, schedule, and stopping rule.

Choose one simple baseline. For a small convex problem, that might be full-batch gradient descent with backtracking or L-BFGS. For a large non-convex problem, it might be momentum SGD with a short learning-rate range test and a fixed budget. Complexity in the baseline makes failures harder to localise.

Store the contract in a machine-readable configuration and serialise the resolved version used by the run. Defaults supplied by code are part of the contract even when they were absent from the user's configuration. Hash or otherwise identify the data snapshot and split membership; a seed cannot reconstruct data that have changed.

Define the trace schema before training. A useful per-update or periodically sampled record contains:

Field Group	Examples	Question Answered
Work	update, epoch, examples, objective calls, elapsed time	How much resource has been spent?
Objective	total, data term, each penalty, validation metric	Which component changed?

Field Group	Examples	Question Answered
Stationarity	gradient, projected-gradient, proximal-gradient, or KKT residual	Is the appropriate first-order condition improving?
Step	learning rate, step norm, parameter norm, momentum norm	Did the computed direction produce a meaningful update?
Stochasticity	batch identity, gradient variance probe, clipping fraction	Is noise or bias dominating?
Curvature	directional curvature, inner residual, trust ratio, rejected pair	Does the local model explain the step?
Feasibility	equality, inequality, complementarity, duality gap	Is progress compatible with the constraints?
Numerical health	non-finite count, loss scale, extreme activation or score	Is arithmetic corrupting the result?

The logging interval can be coarser than the update interval, but event-driven records should capture rejected line-search steps, trust-region changes, clipping, non-finite values, checkpoint selection, and termination. Use a stable schema so method comparisons do not lose fields precisely when a method behaves differently.

9.2 Step 2: Validate the Objective and Derivatives

Test the loss on hand-calculated cases. Check invariants such as non-negativity where appropriate, permutation invariance of a data average, and the expected effect of duplicating or reweighting examples. Compare analytic or automatic derivatives with central finite differences at several deterministic points.

For second-order code, check Hessian symmetry numerically and compare Hessian-vector products with finite differences of gradients. For constraints,

test projections on points inside, outside, and on the boundary. For proximal code, verify the defining optimality condition on small inputs.

Run tests in an order that localises failures:

1. **Scalar cases:** calculate one or two losses by hand, including an extreme score that exercises the stable formula.
2. **Invariances:** permute examples, repartition a full batch, or apply another transformation that should leave the objective unchanged.
3. **Term isolation:** disable each regulariser or constraint term and verify the predicted change in value and derivative.
4. **Directional derivative:** for a fixed unit vector $\mathbf{v}$, compare

$$\frac{F(\theta + h\mathbf{v}) - F(\theta - h\mathbf{v})}{2h}$$

   with $\nabla F(\theta)^\top \mathbf{v}$ over several h values.
5. **Hessian action:** compare a Hessian-vector product with a central difference of gradients.
6. **Update identity:** save parameters before and after one optimiser step and compare the actual difference with the update reconstructed from logged state.

Use relative error with an absolute floor:

$$e_{mathrmrel} = \frac{|a - b|}{\max(1, |a|, |b|)}.$$

A relative error alone is unstable when both quantities are near zero. Expect a U-shaped finite-difference error curve as h decreases: truncation error falls first, then cancellation grows. If no range agrees, suspect the implementation before changing the optimiser.

Stochastic components must be frozen for these tests. Fix the batch and masks, disable data augmentation, and put stateful normalisation in a declared mode. A finite-difference check against two different random computations measures randomness, not a derivative error.

9.3 Step 3: Run Small Problems With Known Answers

Use the rotated quadratic to expose conditioning and step stability. Use a tiny regularised logistic data set for which a trusted full-batch solver and the implemented method can be compared. Overfit a very small training subset when the model should have enough capacity; failure to do so often reveals a pipeline or derivative bug before large-scale tuning begins.

Use a ladder rather than jumping directly from unit tests to the full model:

1. **One-dimensional quadratic:** verifies update signs, step stability, and stopping logic.
2. **Rotated quadratic:** exposes conditioning, momentum stability, preconditioning, and exact Newton behaviour.
3. **Tiny regularised logistic regression:** tests finite-sum scaling, stable loss evaluation, convex convergence, and agreement with a trusted high-accuracy solution.
4. **Synthetic constrained problem:** tests projection or KKT residuals against a solution available by inspection.
5. **Tiny overfit task:** tests whether the complete model and data pipeline can drive training error down when generalisation is irrelevant.
6. **Reduced real task:** retains the actual preprocessing and model code but uses a small deterministic subset, making repeated diagnostic runs cheap.

Each rung introduces one class of complexity. Do not proceed merely because the curve looks plausible; define an acceptance check. On the quadratic, compare with the known minimiser and predicted contraction. On logistic regression, compare objective and gradient with an independent solver or a much tighter deterministic baseline. On the constrained problem, check all KKT residual components. On the tiny overfit task, inspect both the predictions and the loss.

The rotated quadratic can also compare work units. Gradient descent uses one gradient per step. Newton uses a gradient, curvature representation, and linear solve. A fair teaching trace should show iterations and operations separately. Its point is not that Newton is always faster, but that curvature can exchange expensive local computation for fewer outer steps.

The executable Python and R examples in the recurring quadratic and logistic-regression subsections provide the known-answer traces, derivative checks, op-

timisation diagnostics, and held-out validation evidence for these two rungs.

9.4 Step 4: Monitor Quantities That Separate Failures

Record enough information to distinguish instability, noise, conditioning, infeasibility, and overfitting. At a minimum, retain training objective components, validation metrics, gradient norm or a declared proxy, update norm, parameter norm, learning rate, batch identifiers or sampler state, elapsed time, and non-finite counts.

For constrained problems, add primal residuals, dual residuals, and a duality gap when valid. For curvature-aware methods, retain line-search evaluations, trust-region radius and acceptance ratio, inner-solve residuals, or curvature-pair acceptance. For distributed training, add worker count, effective batch size, gradient accumulation, and communication or staleness metrics.

The table maps common signatures to the next discriminating test.

Observed Signature	Plausible Causes	Next Check
Loss becomes non-finite	Excessive step, unstable loss formula, overflow, corrupt input	Re-run the first failing batch with anomaly checks and a smaller step
Loss oscillates with growing amplitude	Step or momentum outside stability region	Reduce both separately; inspect update-to-parameter ratio
Loss is flat and gradient is large	Step too small, frozen parameters, update not applied	Compare computed update with actual parameter difference
Loss and gradient are both nearly flat	Plateau, saturation, poor scaling, or convergence	Check directional curvature, rescale inputs, and compare with a trusted solver
Training improves while validation degrades	Overfitting, split mismatch, or validation adaptation	Inspect per-group metrics and the pre-declared stopping rule

Observed Signature	Plausible Causes	Next Check
Constraint residual stays large	Infeasibility, weak penalty, wrong projection, or bad scaling	Solve a feasibility problem and check units
Different seeds give qualitatively different results	Multiple basins, data-order sensitivity, or nondeterministic kernels	Increase repetitions and isolate each randomness source
Full-batch method works but mini-batch method fails	Estimator scaling, sampler bias, excessive noise, or stateful batch effects	Compare averaged mini-batch gradients with the full gradient

When the trace suggests more than one cause, run the smallest discriminating experiment:

- **Large gradient, no parameter movement:** reconstruct one update from optimiser state. If it is non-zero but parameters do not change, inspect frozen flags, update application, mixed-precision overflow handling, and checkpoint restoration.
- **Small gradient, poor objective:** compare directional curvature and restart from a perturbed point. A positive-definite local Hessian suggests a local minimum; negative curvature suggests a saddle; uniformly tiny curvature suggests a plateau or scaling problem.
- **Mini-batch failure after full-batch success:** average many mini-batch gradients at the same fixed point and compare with the full gradient. A persistent difference indicates weighting, sampling, stateful layers, or regularisation scaling rather than mere variance.
- **Constraint violation with falling penalised objective:** plot the original objective and violation separately. The penalty may be too weak, the constraints may be infeasible, or the solver may be trading a large feasibility loss for a small objective gain hidden by scale.
- **Adaptive method takes rare huge steps:** inspect denominator quantiles and clipping events by parameter block. Increasing the stabiliser may hide, rather than solve, a malformed gradient scale.
- **Validation instability across seeds:** freeze the trained checkpoints and

recompute metrics on identical examples. This separates training variation from evaluation sampling or non-deterministic inference.

Use *counterfactual replays* when possible. Save the state immediately before a suspicious update, then replay that same batch with one controlled change: half the step, no momentum, full precision, no clipping, or an exact full gradient. Replaying from a common state is more informative than comparing two long runs that diverged thousands of updates earlier.

Residual plots should use scales that expose the relevant regime. A log vertical axis reveals geometric contraction and residual floors. Plot objective gap only when a credible reference value exists; subtracting the smallest observed value can falsely make the best run look certified. For stochastic traces, show a lightly smoothed trend together with raw or quantile information and state the smoothing window.

A diagnostic should change one hypothesis at a time. Lowering the learning rate, changing the optimiser, adding clipping, changing batch size, and changing initialisation in one run may improve the curve while teaching nothing about the original failure.

9.5 Step 5: Compare Methods Fairly

Define the budget before looking at results: gradient evaluations, examples processed, objective calls, Hessian-vector products, wall-clock time on fixed hardware, energy, or another resource. Report at least one method-independent unit such as examples processed alongside wall-clock time.

Tune each method over a defensible search space. A single default for one method compared with a tuned schedule for another is not evidence about the method families. Use the same data splits and acceptance rule, preserve all trials, and report variation across seeds. If one method uses more memory or parallel workers, include that resource difference.

Optimisation performance should include final training objective, time-to-threshold, and stability. Machine-learning performance should include held-out metrics and uncertainty appropriate to the sampling design. Neither set replaces the other.

Treat a method comparison as an experiment with controlled factors. Separate

three questions:

1. Which method reaches a declared optimisation threshold with the least resource?
2. Which complete training procedure produces the best held-out result under a fixed selection budget?
3. Which procedure is most reliable under perturbations in seed, data order, precision, or hardware?

These questions can have different winners. The first requires a method-independent optimisation criterion. The second includes schedules, regularisation, and checkpoint selection. The third requires repeated runs and a failure definition.

Use paired trials where appropriate: the same split, preprocessing, initial model function, and evaluation checkpoints. Exact parameter matching may be impossible across methods with different state, but nuisance variables should be aligned deliberately. Do not align a source of randomness if its distribution is part of the method being evaluated.

Report a resource vector rather than one scalar when costs differ materially: elapsed time, examples processed, objective calls, gradient calls, Hessian-vector products, peak memory, and worker count. Wall-clock measurements need warm-up policy and fixed hardware. Operation counts remain portable but do not capture kernel efficiency or communication.

Hyperparameter search cost belongs to the comparison. If method A needed 100 trials and method B needed 10, comparing only the best final runs hides most of the resource and selection bias. Report search spaces, trial counts, failed trials, and the rule used to choose the winner.

9.6 Step 6: Decide, Stop, and Preserve Evidence

Stop for one of four explicit reasons: a convergence certificate reached its tolerance, validation policy selected a checkpoint, a resource budget was exhausted, or a failure condition fired. Record which reason applied. “Training finished” is not specific enough.

Persist the parameters, full optimiser state, schedule state, random-number state, data identity, code revision, environment, logs, and configuration. Verify

that resuming from a checkpoint reproduces the next few updates within the expected determinism limits. Preserve the best validation checkpoint separately from the final optimiser iterate if the policy distinguishes them.

The final report should state unresolved issues. Examples include a noisy stationarity estimate, a constraint residual above tolerance, a validation result sensitive to one seed, or a Hessian approximation that was not independently checked. An explicit limitation is more useful than an unsupported convergence claim.

Write a termination record with these fields:

- termination class: certificate, validation policy, budget, or failure;
- triggering quantity, threshold, and number of consecutive checks required;
- final and best-seen objective components;
- final stationarity or KKT residuals with their estimation method;
- selected checkpoint and whether it differs from the final iterate;
- total declared work units and elapsed time;
- numerical anomalies, rejected steps, clipping rate, and failed inner solves;
- unresolved assumptions or diagnostics.

For stochastic stopping, avoid firing on one favourable noisy measurement. Use a fixed diagnostic sample, repeated estimates, or a confidence rule declared in advance. The stopping calculation must not silently become another repeatedly queried validation set that drives model selection.

When a run fails, preserve a *failure packet*: the last good checkpoint, first bad checkpoint, optimiser and random state, batch identifiers, recent trace window, resolved configuration, code revision, and environment. A reproducible failure is an asset. Restarting immediately with several simultaneous changes destroys the evidence needed to identify the cause.

Checkpoint validation is part of the workflow. Resume a saved state and verify the next few updates against an uninterrupted run within the expected determinism limits. A file that reloads parameters but loses momentum, adaptive moments, schedule counters, gradient-scaler state, or sampler position is an inference checkpoint, not an exact training-resume checkpoint.

9.7 Method-Selection Checklist

Use the following order when choosing a method:

1. Formulate and test the objective, constraints, and data split.
2. Classify smoothness, convexity, stochasticity, scale, and exploitable structure.
3. Estimate the cost of a gradient, objective evaluation, Hessian-vector product, and projection or proximal map.
4. Choose the simplest method whose assumptions and memory needs fit.
5. Add line search, momentum, preconditioning, curvature, or adaptivity only for an observed need.
6. Select stopping and failure criteria before the long run.
7. Compare at a declared budget and retain enough state to reproduce the decision.

For small regularised logistic regression, L-BFGS or damped Newton is a strong baseline. For a massive sparse convex problem, mini-batch, coordinate, or variance-reduced first-order methods may dominate. For large neural training, momentum or adaptive diagonal methods with a declared schedule are practical starting points. For smooth moderate constrained problems, sequential quadratic programming, interior-point, or augmented-Lagrangian methods may exploit structure better than penalty-only SGD.

Close every optimisation study with a compact decision record:

> **Problem:** exact objective, constraints, data, and parameterisation. **Method:** update equations, state, schedule, precision, and safeguards. **Assumptions:** those required by the stated guarantee, with evidence or an explicit "not verified". **Budget:** operations, data exposure, time, memory, and search trials. **Outcome:** objective, appropriate residual, feasibility, held-out metrics, and variation across runs. **Termination:** certificate, validation selection, budget exhaustion, or failure. **Limitations:** unresolved numerical, statistical, or operational concerns.

This record is more valuable than naming the winning optimiser. It lets another reader decide whether the result transfers to a different data scale, tolerance, constraint, or hardware setting.

10 Further Reading

The sources below provide full proofs, broader algorithm catalogues, and primary discussions of methods mentioned in the primer. The annotations identify their role.

- Stephen Boyd and Lieven Vandenberghe, *Convex Optimization*, Cambridge University Press (2004). A freely available, proof-based treatment of convex sets, duality, optimality conditions, unconstrained methods, and interior-point methods.
- Jorge Nocedal and Stephen J. Wright, *Numerical Optimization*, second edition, Springer (2006). The main reference for line searches, trust regions, Newton and quasi-Newton methods, constrained optimisation, and their convergence theory.
- Yurii Nesterov, *Lectures on Convex Optimization*, second edition, Springer (2018). A rigorous source for complexity bounds, accelerated first-order methods, and condition-number dependence.
- Léon Bottou, Frank E. Curtis, and Jorge Nocedal, "Optimization Methods for Large-Scale Machine Learning", *SIAM Review* 60 (2018), 223–311. Connects stochastic-gradient theory, noise, batching, and second-order ideas to large-scale learning.
- Neal Parikh and Stephen Boyd, "Proximal Algorithms", *Foundations and Trends in Optimization* 1 (2014), 127–239. Develops proximal operators, splitting methods, and applications to structured non-smooth objectives.
- John Duchi, Elad Hazan, and Yoram Singer, "Adaptive Subgradient Methods for Online Learning and Stochastic Optimization", *Journal of Machine Learning Research* 12 (2011), 2121–2159. The primary AdaGrad analysis and a useful account of geometry-aware diagonal adaptation.
- Diederik P. Kingma and Jimmy Ba, "Adam: A Method for Stochastic Optimization", *International Conference on Learning Representations* (2015). Defines Adam and its moment estimates; read its guarantee together with later corrections.
- Sashank J. Reddi, Satyen Kale, and Sanjiv Kumar, "On the Convergence of Adam and Beyond", *International Conference on Learning Representations* (2018). Gives a convex non-convergence example for Adam and develops the AMSGrad response.
- Ilya Loshchilov and Frank Hutter, "Decoupled Weight Decay Regulariza-

tion”, *International Conference on Learning Representations* (2019). Separates an ℓ_2 objective penalty from decoupled parameter decay under adaptive methods.

- Yann N. Dauphin, Razvan Pascanu, Caglar Gulcehre, Kyunghyun Cho, Surya Ganguli, and Yoshua Bengio, “Identifying and Attacking the Saddle Point Problem in High-Dimensional Non-Convex Optimization”, *Advances in Neural Information Processing Systems* 27 (2014). A primary empirical and theoretical argument for studying saddle structure in high-dimensional learning objectives.
- James Bergstra and Yoshua Bengio, “Random Search for Hyper-Parameter Optimization”, *Journal of Machine Learning Research* 13 (2012), 281–305. Establishes random search as a serious outer-loop baseline and explains why grids waste trials when only some hyperparameters matter.